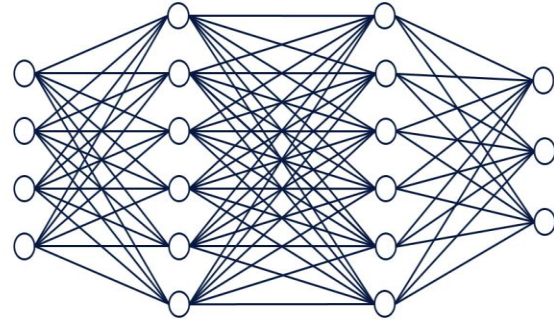
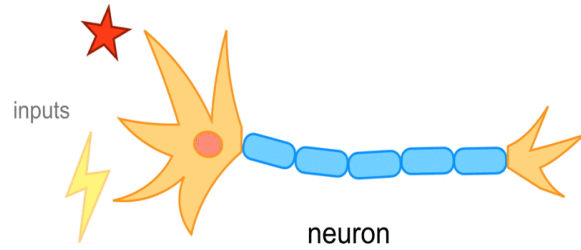


Introduction to Neural Networks



Artificial neural networks (ANNs)

Other terms/names

- Connectionist
- Parallel distributed processing
- Neural computation
- Adaptive networks..

History of ANNs

- 1943-McCulloch & Pitts are generally recognised as the designers of the first neural network
- 1949-First learning rule
- 1969-Minsky & Papert - perceptron limitation - Death of ANN
- 1980's - Re-emergence of ANN - multi-layer networks

ANN

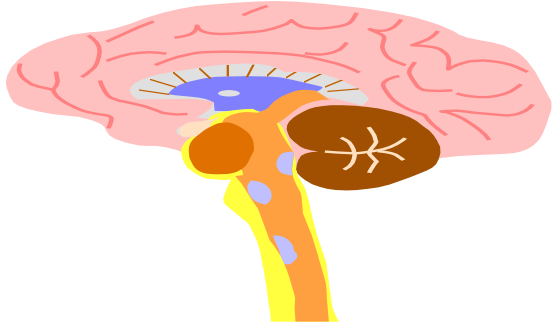
- **Artificial neural networks (ANNs)** are computing systems vaguely inspired by the biological neural networks
- **An ANN** is based on a collection of connected units or nodes called artificial neurons which loosely model the neurons in a biological brain.



ANN

- Each connection, like the **synapses** in a biological brain, can **transmit a** signal from one **artificial neuron to another**.
- An artificial neuron that **receives a signal** can process it and then signal additional artificial neurons connected to it.

Brain and Machine

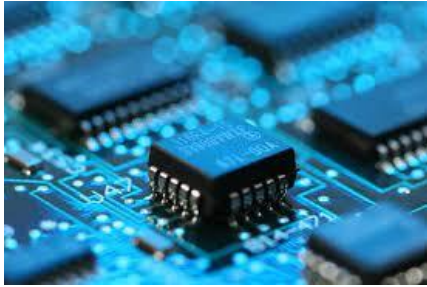


- **The Brain**
 - Pattern Recognition
 - Association
 - Complexity
 - Noise Tolerance

- **The Machine**
 - Calculation
 - Precision
 - Logic



The contrast in architecture



- The Von Neumann architecture uses a single processing unit;
 - Tens of millions of operations per second
 - Absolute arithmetic precision

- The brain uses many slow unreliable processors acting in parallel



Computers vs. Neural Networks

“Standard” Computers

- One CPU
- Fast processing units
- Reliable units
- Static infrastructure

Neural Networks

Highly parallel processing

Slow processing units

Unreliable units

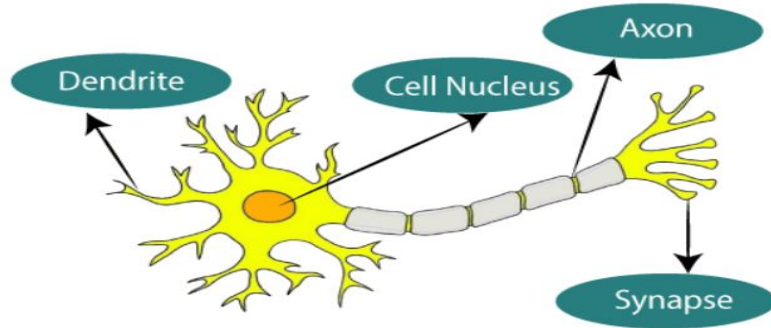
Dynamic infrastructure

Why Artificial Neural Networks?

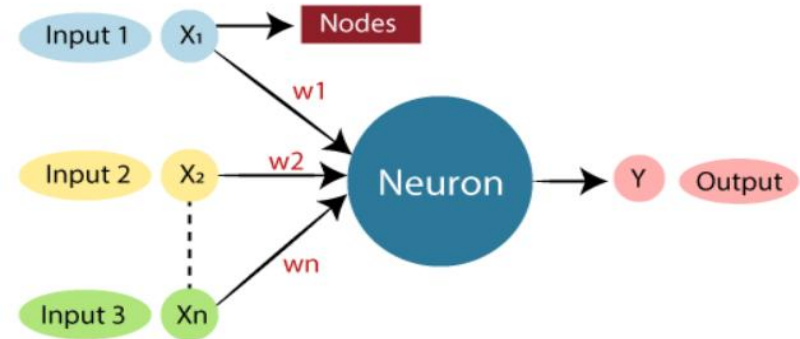
- There are two basic reasons why we are interested in building artificial neural networks (ANNs):
- **Technical viewpoint:** Some problems such as **character recognition** or the prediction of future states of a system require **massively parallel and adaptive processing**.
- **Biological viewpoint:** ANNs can be used to **replicate and simulate** components of the human (or animal) brain, thereby giving us insight into **natural information processing**.

Biological Neural Network Vs Artificial Neural Network

Biological Neural Network



Artificial Neural Network

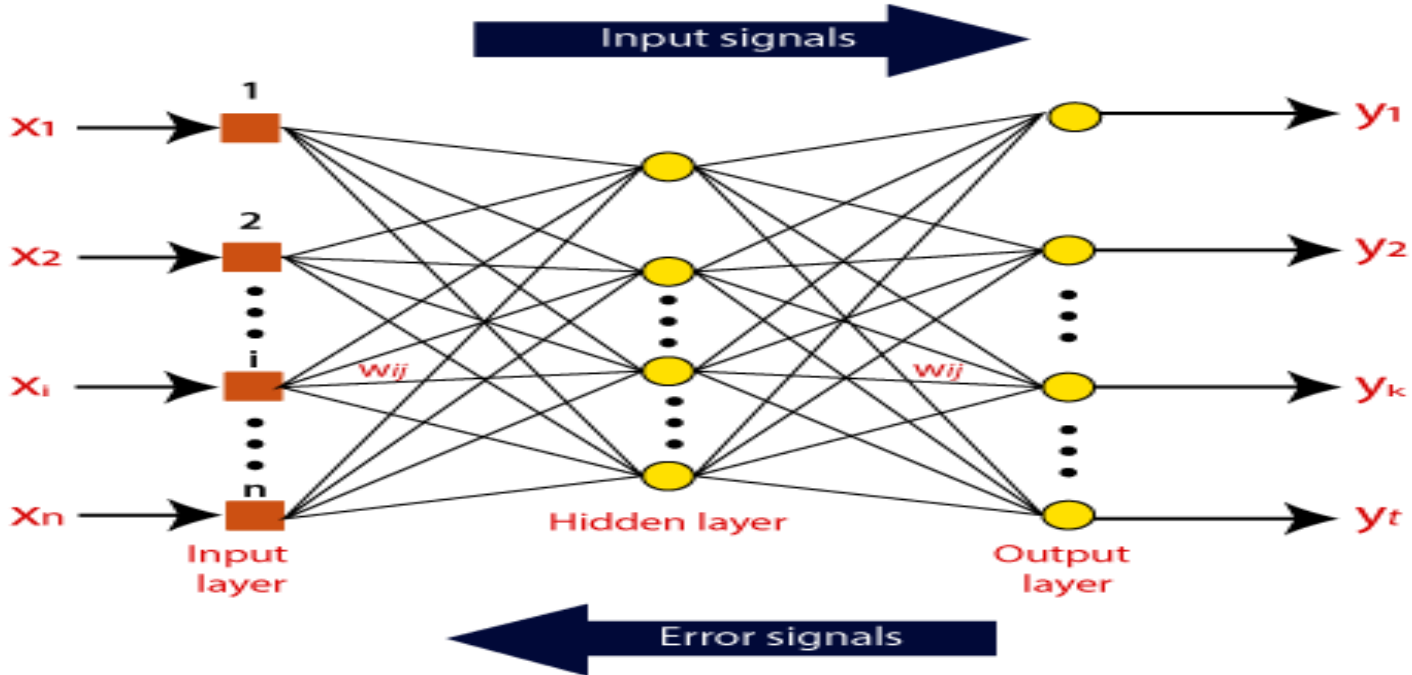


Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

ANN

- The “building blocks” of neural networks are the **neurons**.
 - In technical systems, we also refer to them as **units** or **nodes**.
- Basically, each **neuron**
 - receives **input** from many other neurons.
 - changes its internal state (**activation**) based on the current input.
 - sends **one output signal** to many other neurons, possibly including its input neurons (recurrent network).

The Architecture of ANN



ANN

- In common ANN implementations, the signal at a connection between artificial neurons is **a real number**, and the output of each artificial neuron is computed by some **non-linear function of the sum of its inputs**.
- The connections between artificial neurons are called '**edges**'.

ANN

- Artificial **neurons** and **edges** typically have **a weight** that adjusts as learning proceeds.
- The weight **increases or decreases** the strength of the signal at a connection.
- Typically, artificial neurons are **aggregated into layers**.

ANN

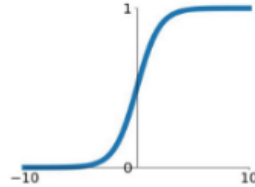
- Different layers may perform different kinds of **transformations** on their inputs.
- Signals travel from the **first layer (the input layer)**, to the **last layer (the output layer)**, possibly after traversing the layers multiple times.

ANN

Activation Functions

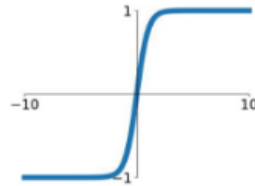
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



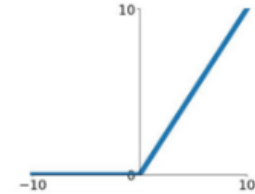
tanh

$$\tanh(x)$$



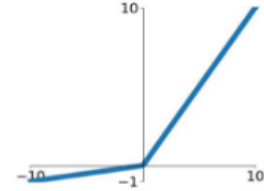
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

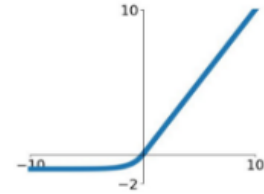


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Advantages of ANN

- Parallel processing
- Storing data on the entire network
- Capability to work with incomplete knowledge
- Memory distribution
- Fault tolerance

Disadvantages of ANN

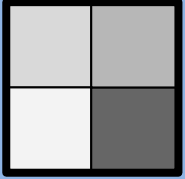
- Hardware Dependence
- Understanding the network's operation
- Assured network structure
- Difficulty in presenting the issue to the network
- The network's lifetime is unknown

Applications ANN

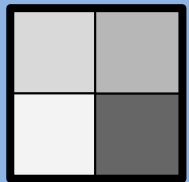
- Image Processing and Character recognition
- Forecasting
- Stock Market Prediction.
- Social Media.
- Aerospace.
- Defence.
- Healthcare.
- Signature Verification and Handwriting Analysis

Example

A four pixel image



Categorize images



solid



vertical



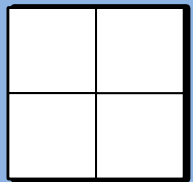
diagonal



horizontal



Categorize images



solid



vertical



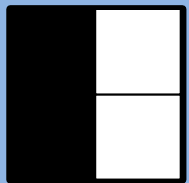
diagonal



horizontal



Categorize images



solid



vertical



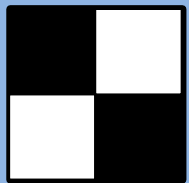
diagonal



horizontal



Categorize images



solid



vertical



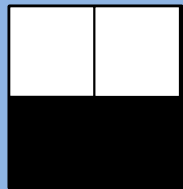
diagonal



horizontal



Categorize images



solid



vertical



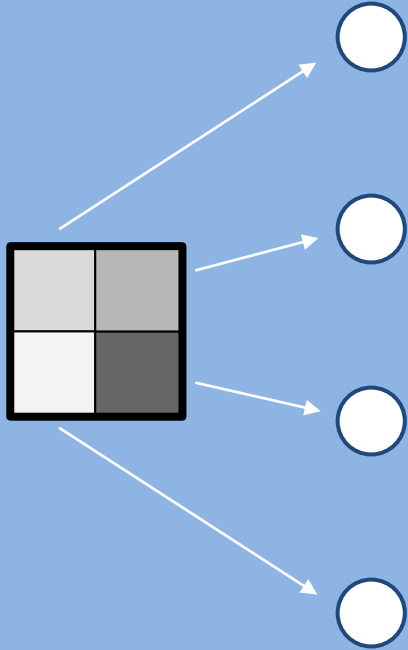
diagonal



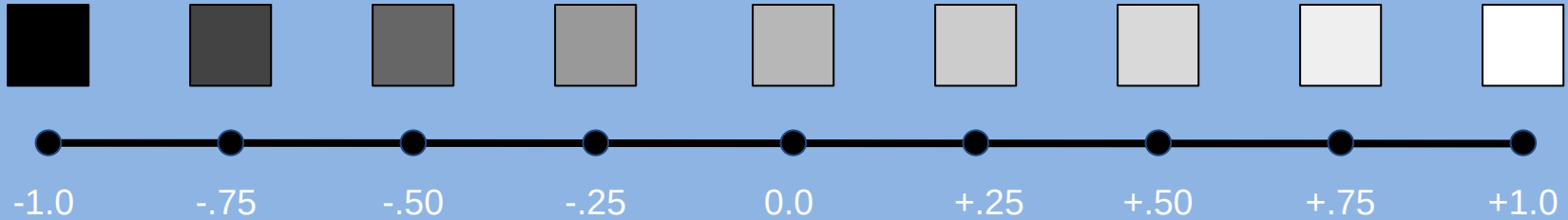
horizontal



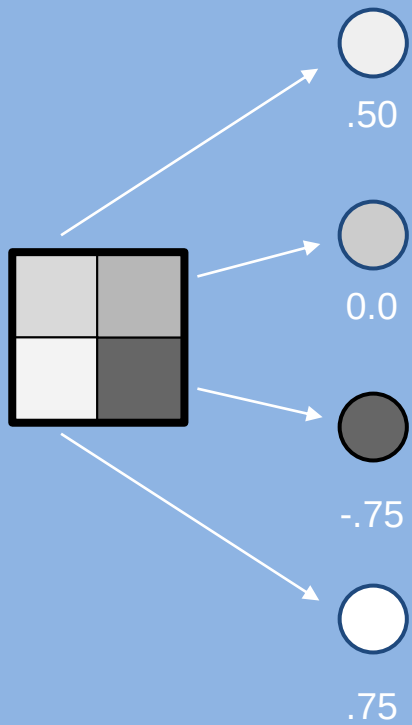
Input neurons



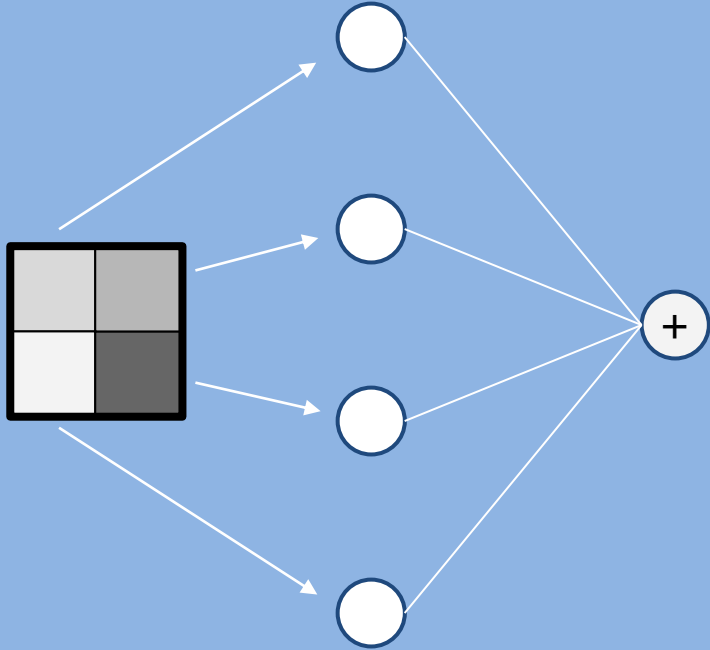
Pixel brightness



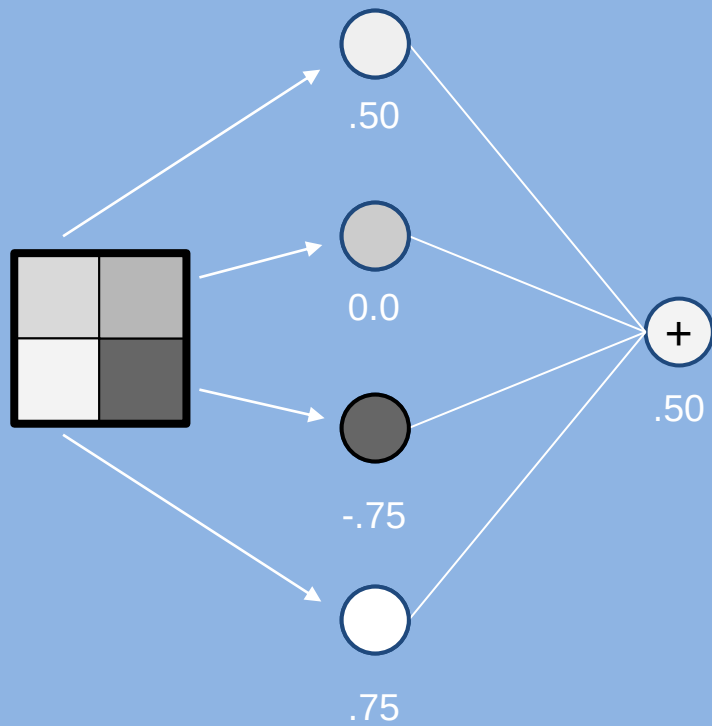
Input vector



A neuron

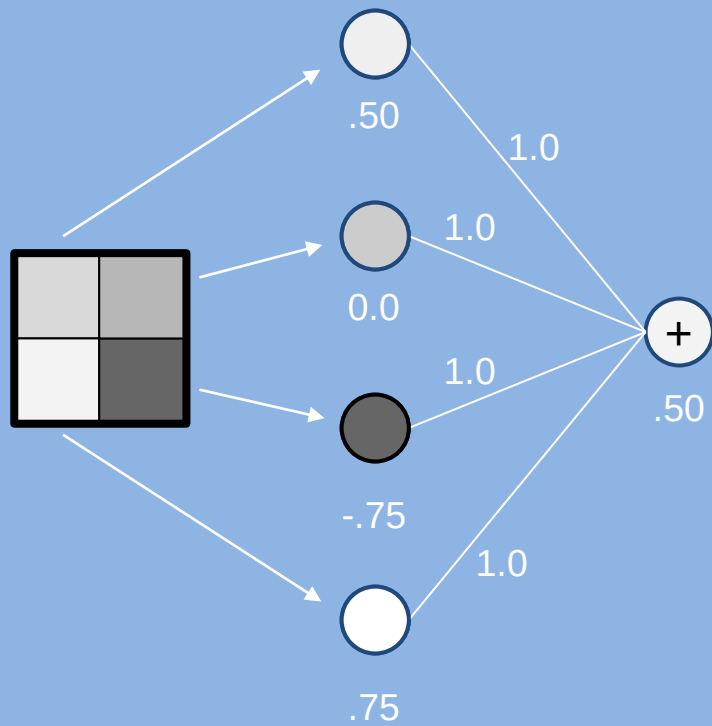


Sum all the inputs



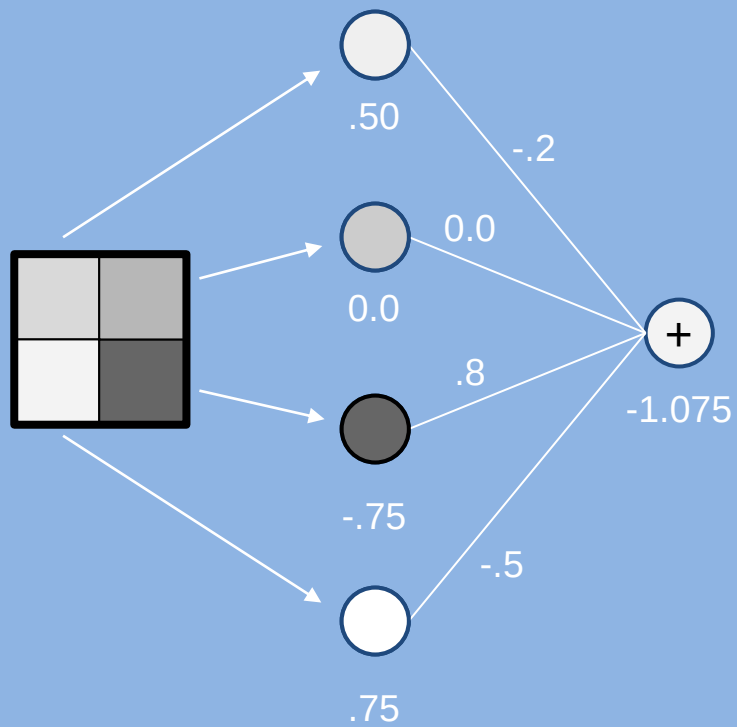
$$\begin{array}{r} .50 \\ 0.00 \\ -.75 \\ + \quad .75 \\ \hline .50 \end{array}$$

Weights



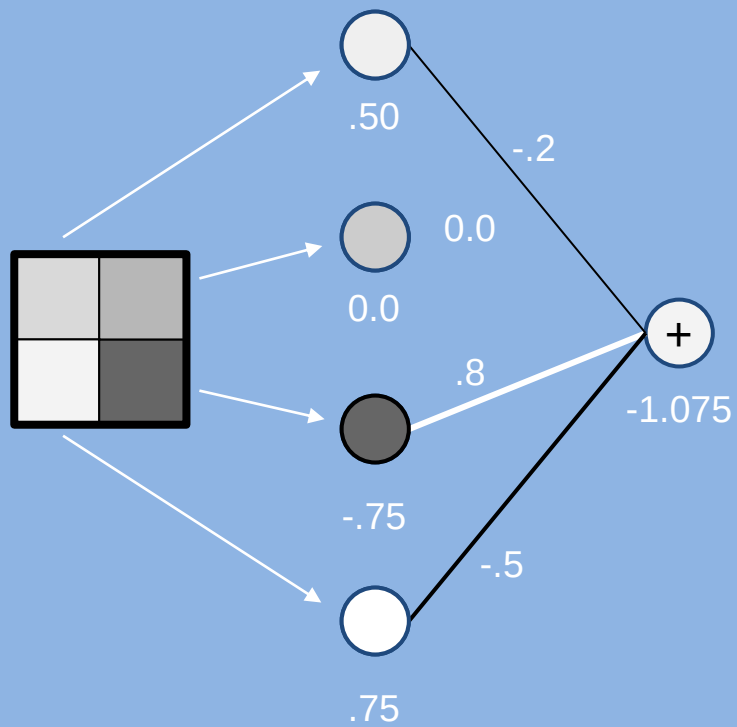
$$\begin{array}{r} .50 \times 1.0 \\ 0.00 \times 1.0 \\ -.75 \times 1.0 \\ + \quad .75 \times 1.0 \\ \hline .50 \end{array}$$

Weights



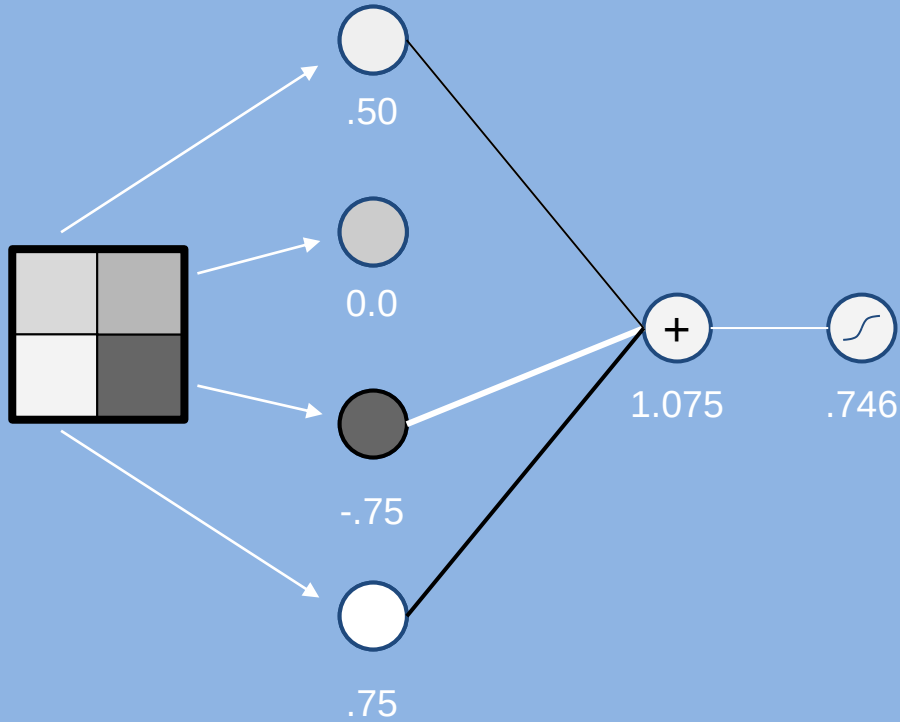
$$\begin{array}{r} .50 \times -.2 \\ 0.00 \times 0.0 \\ -.75 \times .8 \\ + \quad .75 \times -.5 \\ \hline -1.075 \end{array}$$

Weights

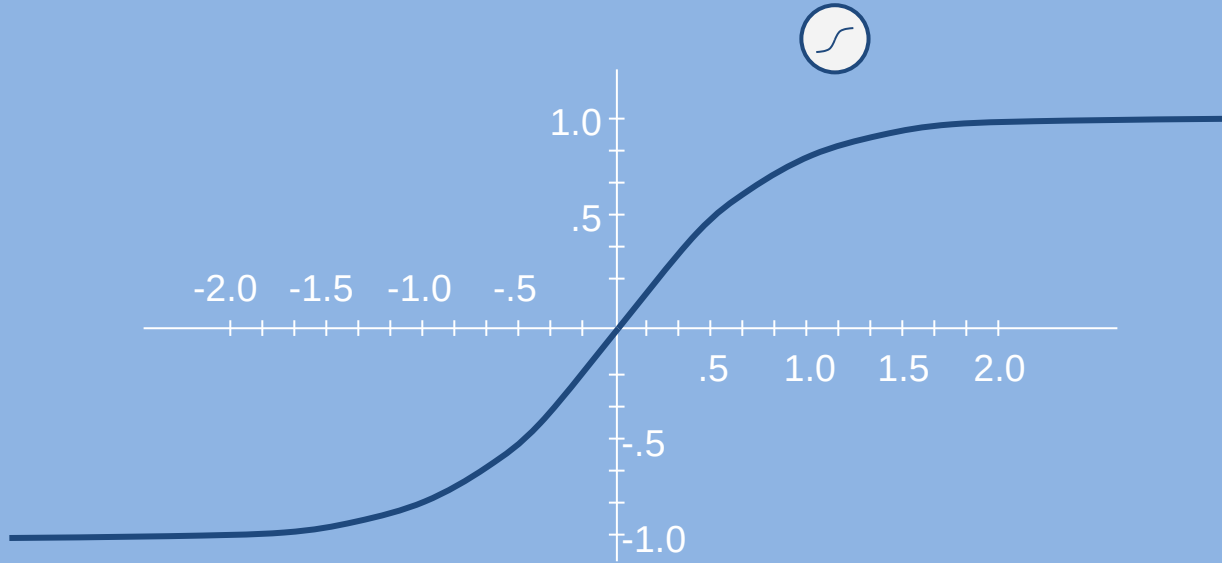


$$\begin{array}{rcl} .50 & \times & -.2 \\ 0.00 & \times & 0.0 \\ -.75 & \times & .8 \\ + & .75 & \times & -.5 \\ \hline & & -1.075 \end{array}$$

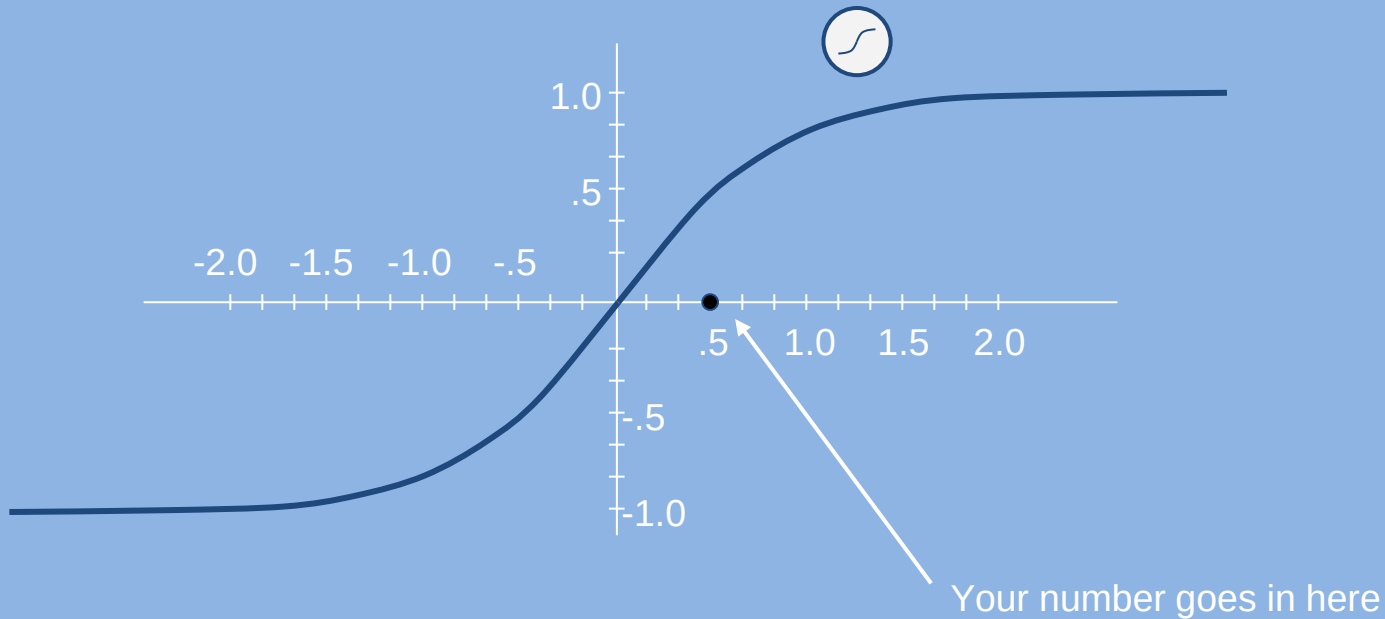
Squash the result



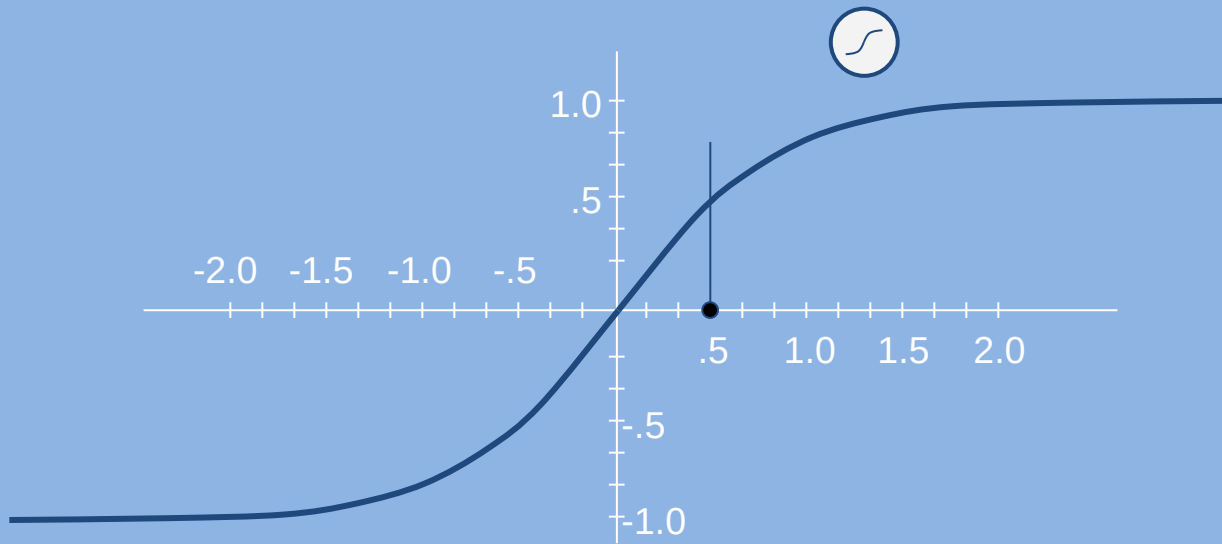
Sigmoid squashing function



Sigmoid squashing function

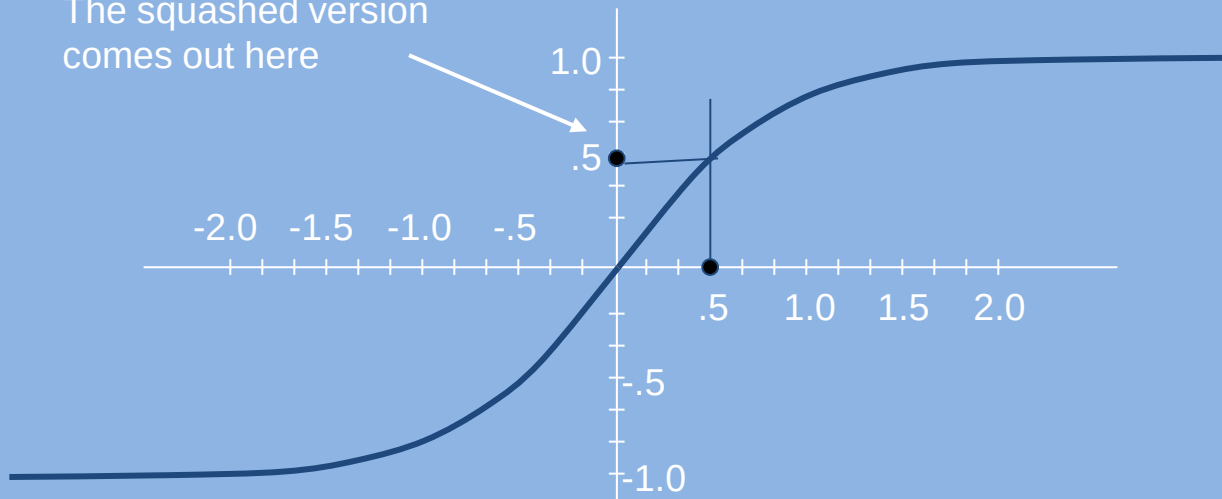


Sigmoid squashing function

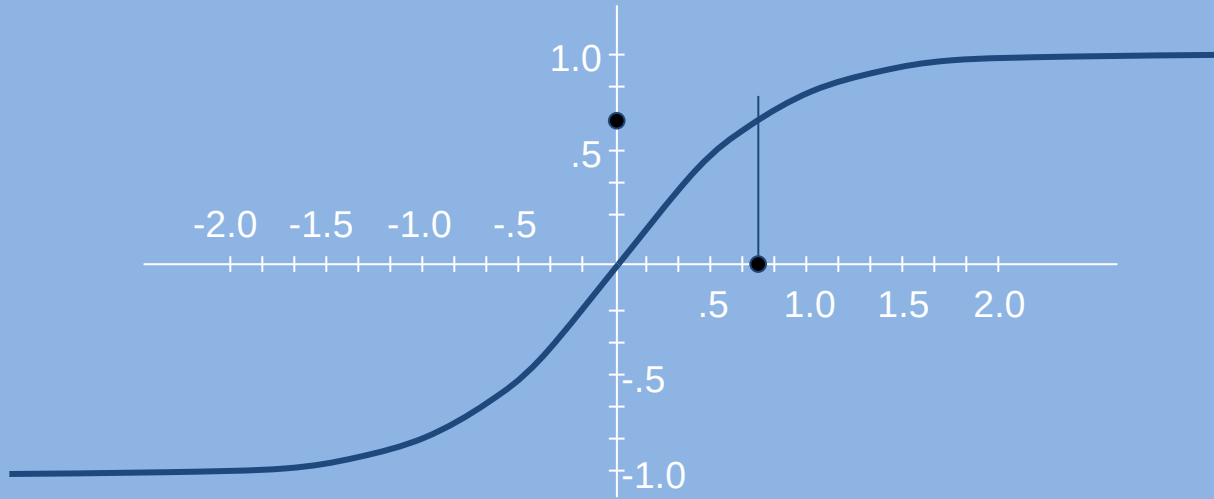


Sigmoid squashing function

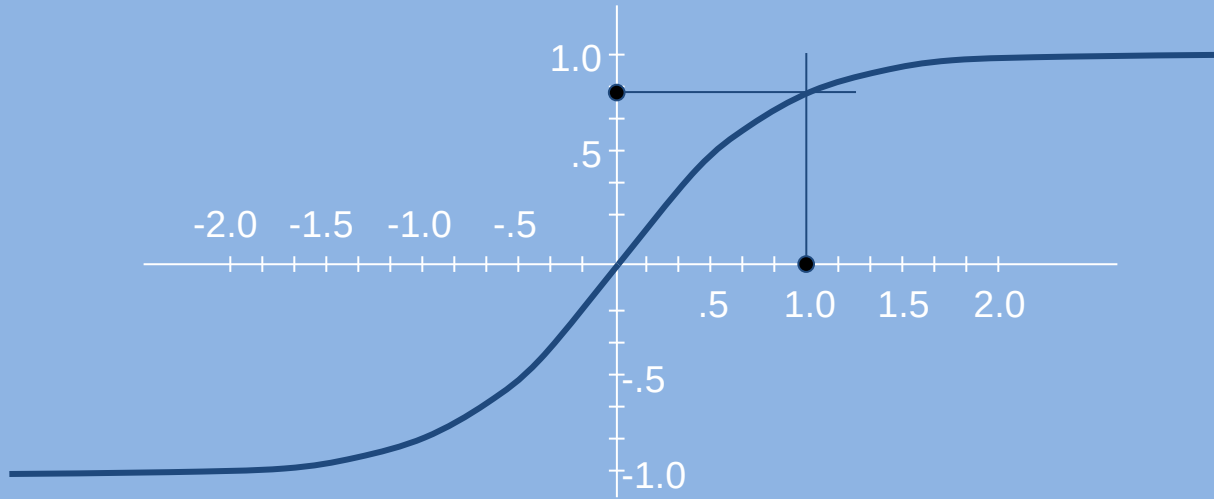
The squashed version
comes out here



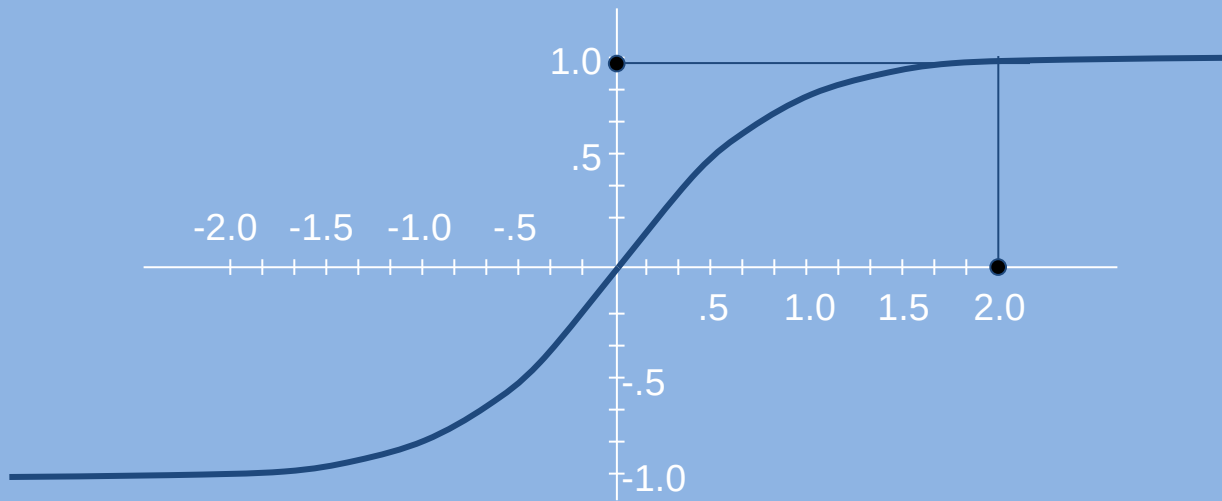
Sigmoid squashing function



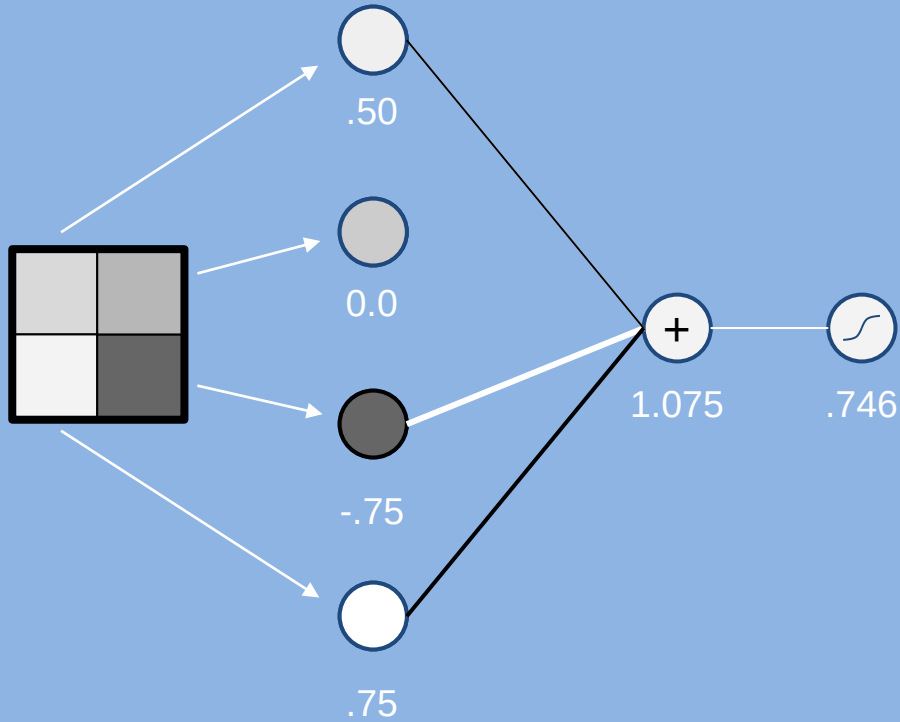
Sigmoid squashing function



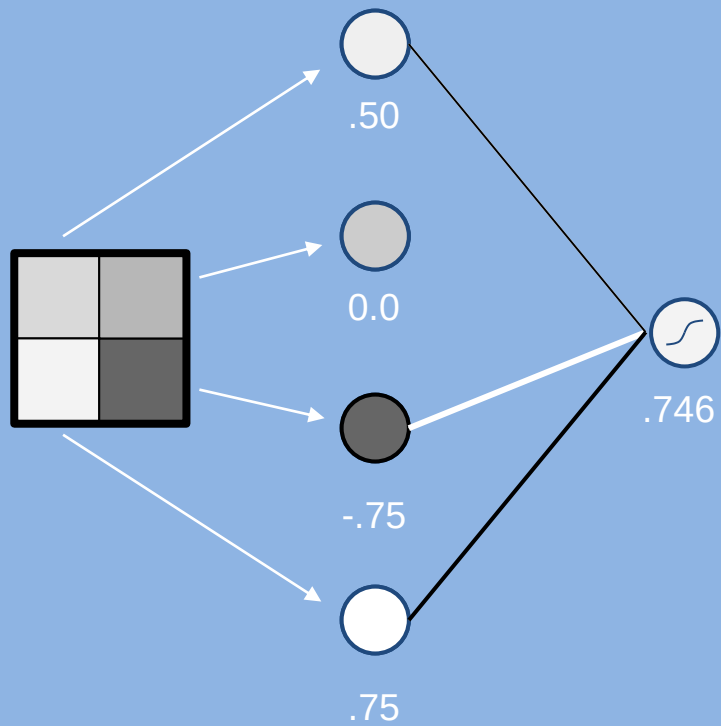
the answer stays between -1 and 1.



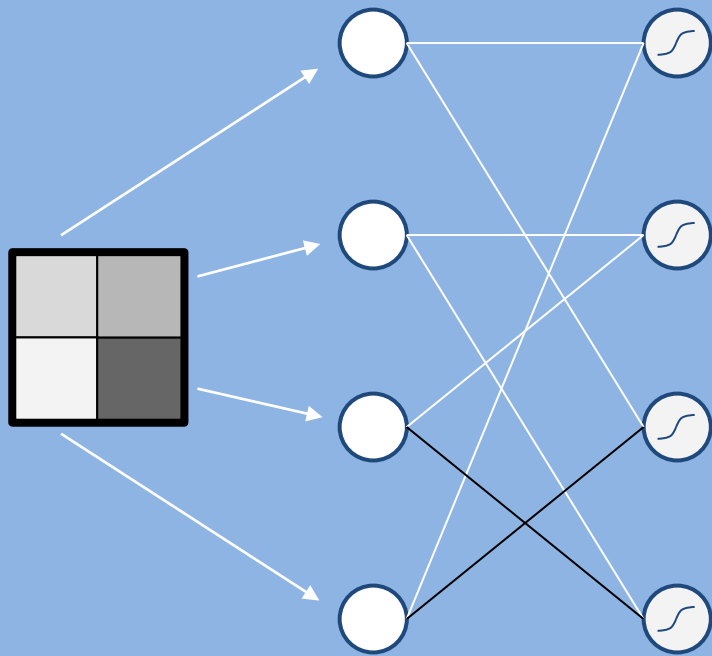
Squash the result



Weighted sum-and-squash neuron

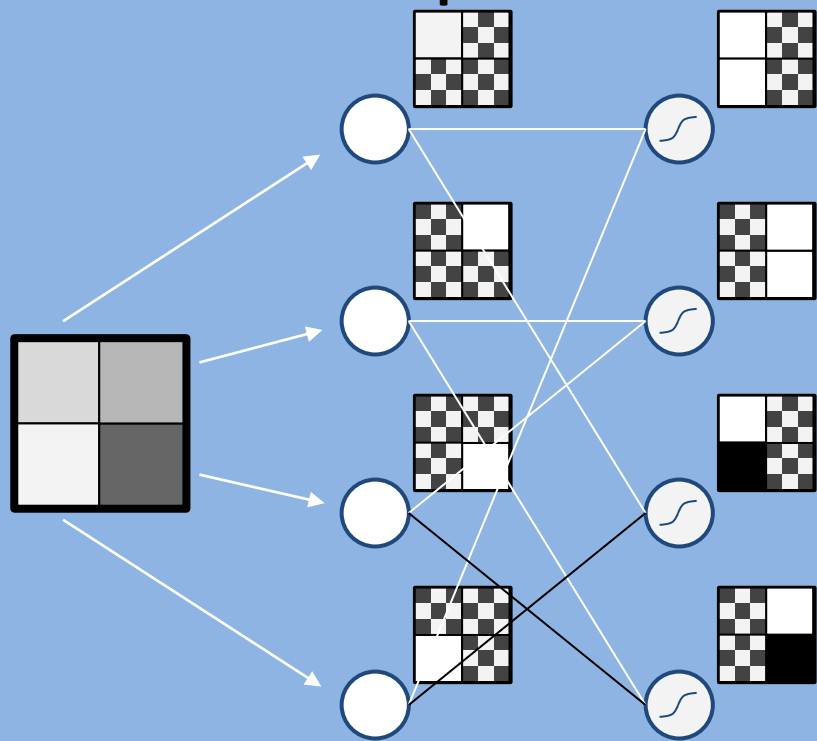


Make lots of neurons, identical except for weights

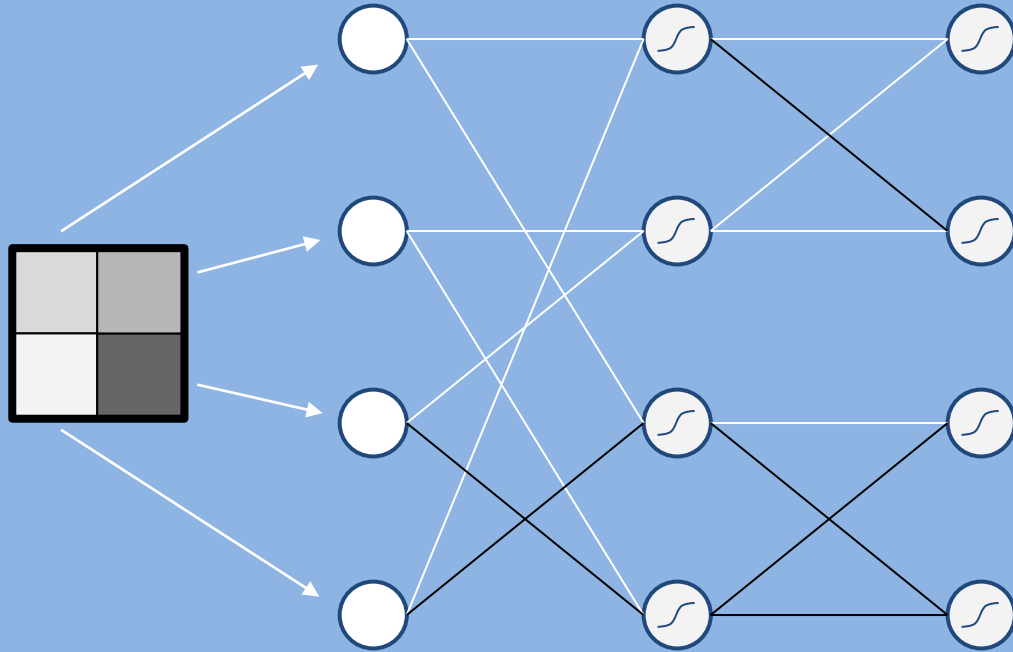


To keep our picture clear, weights will either be
1.0 (white)
-1.0 (black) or
0.0 (missing)

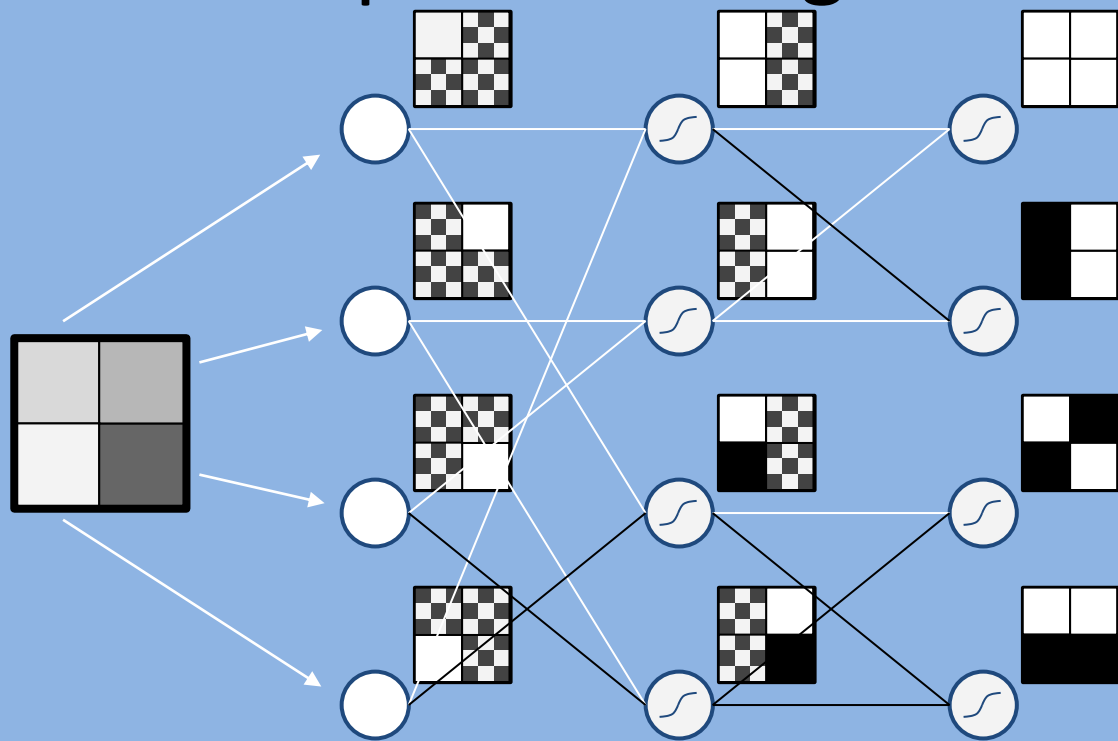
Receptive fields get more complex



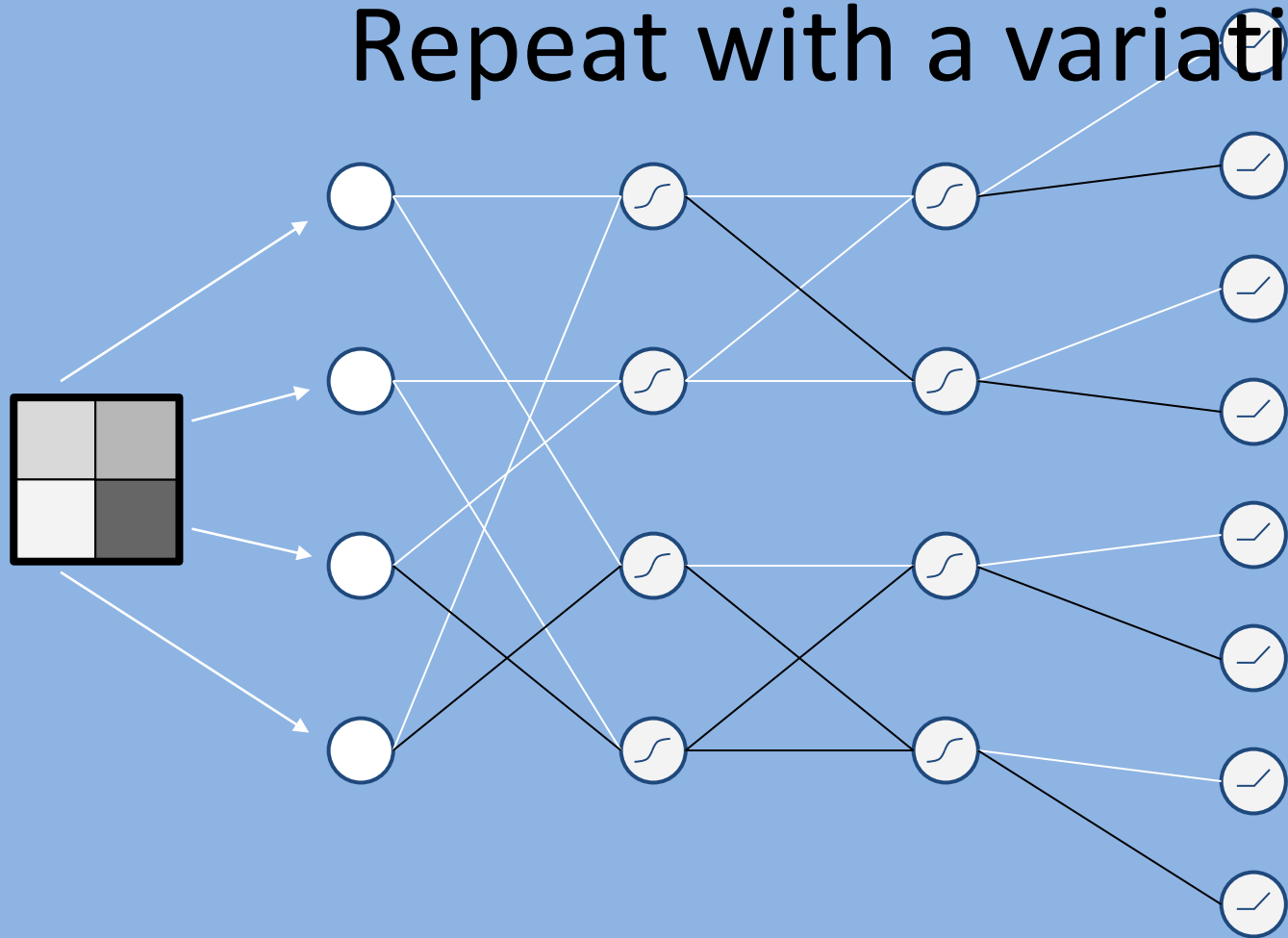
Repeat for additional layers



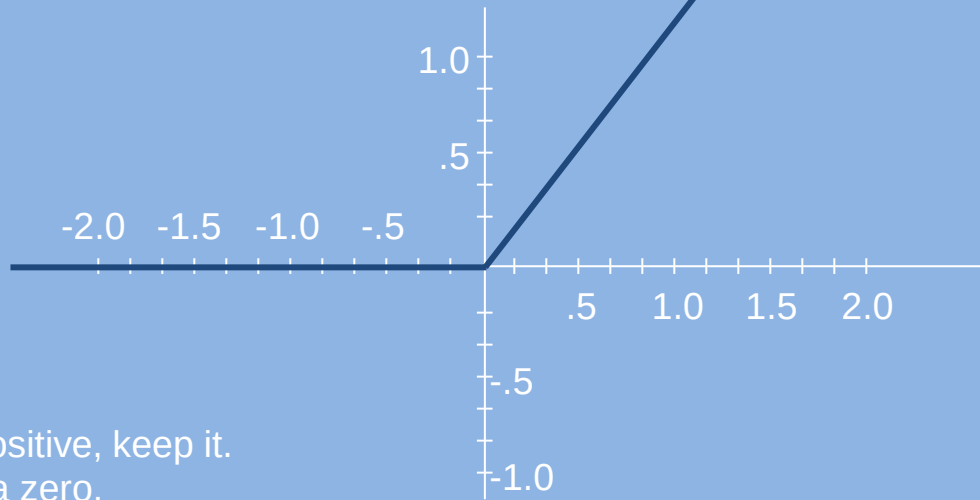
Receptive fields get still more complex



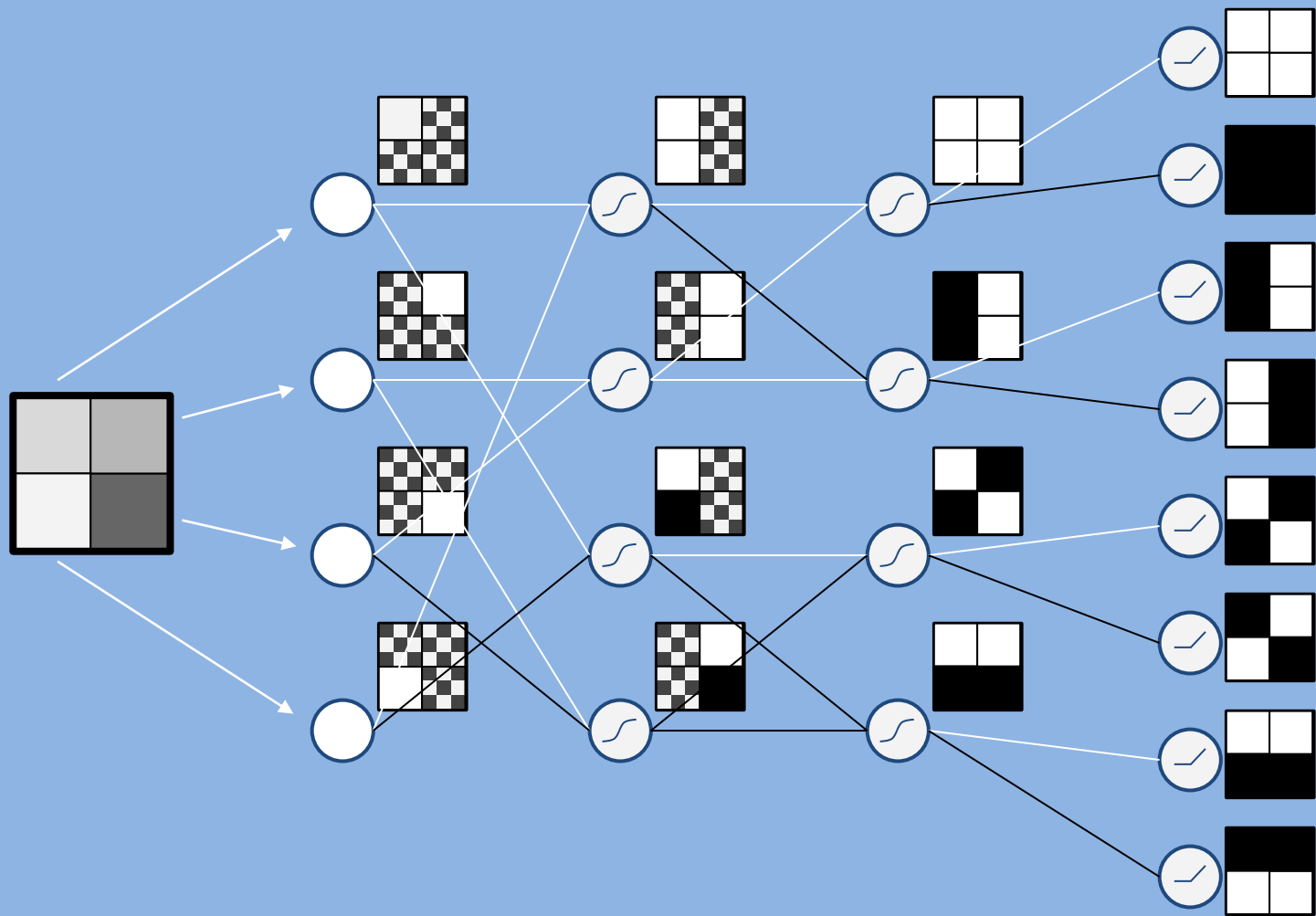
Repeat with a variation

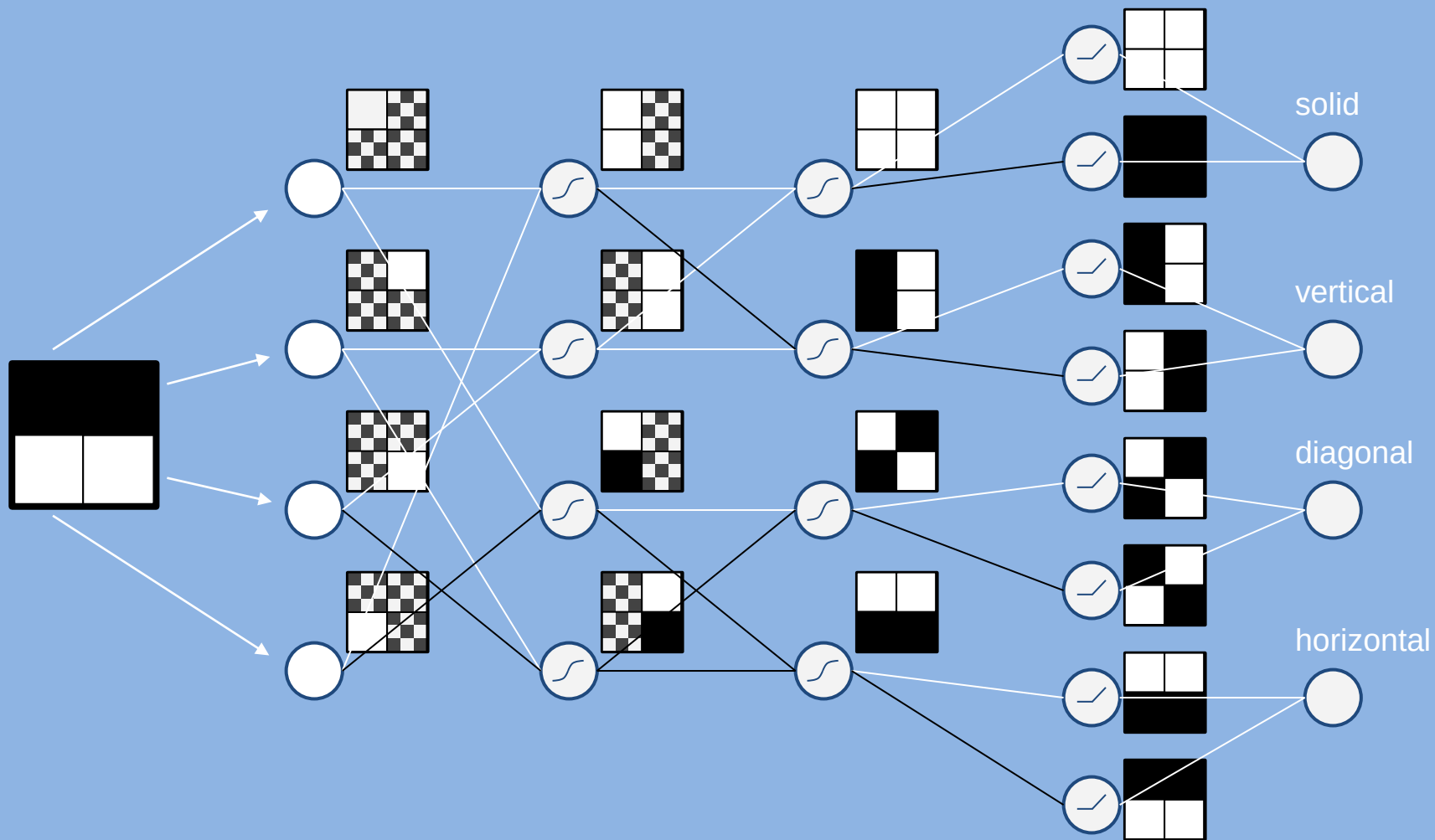


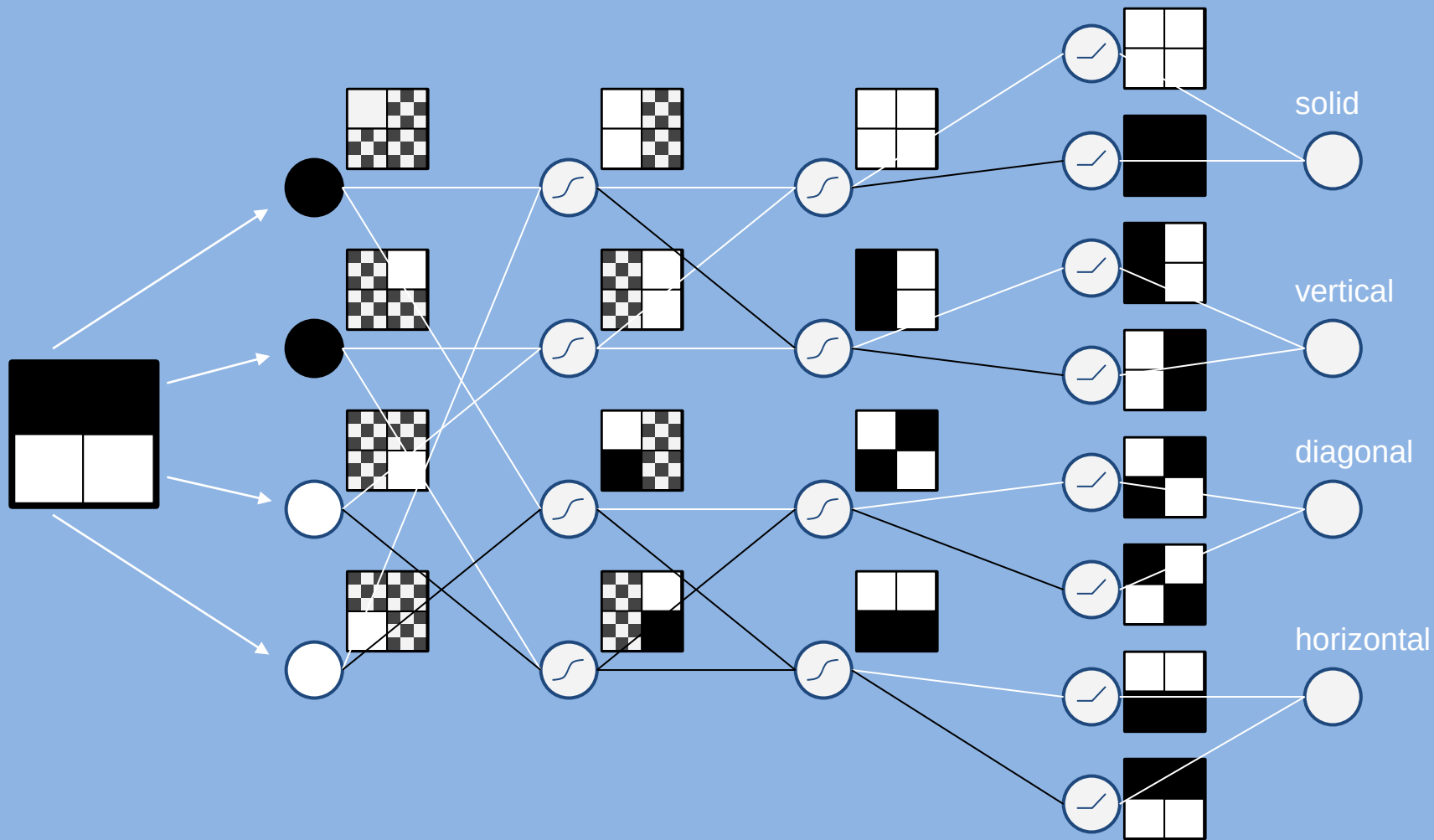
Rectified linear units (ReLU)

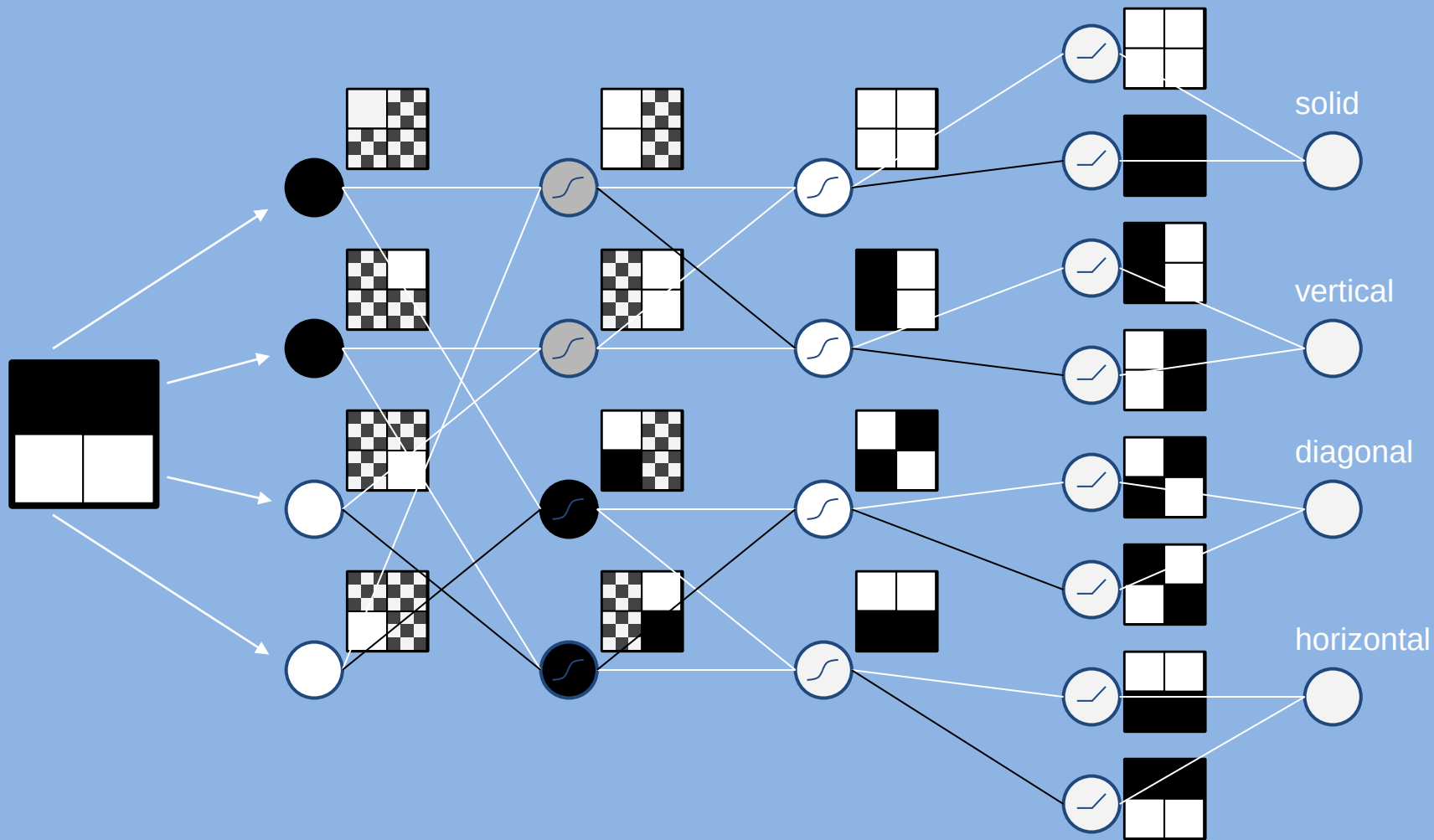


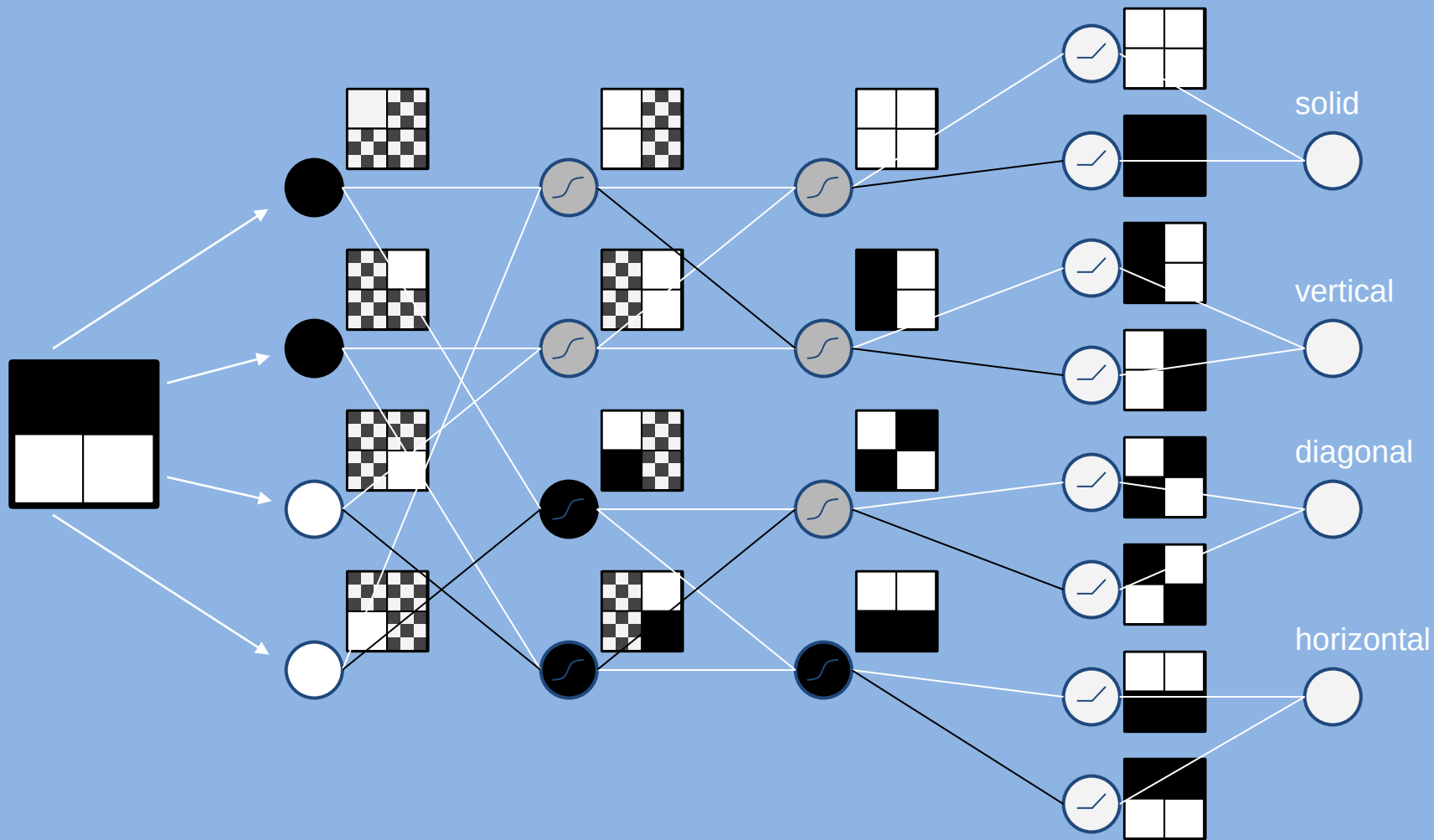
If your number is positive, keep it.
Otherwise you get a zero.

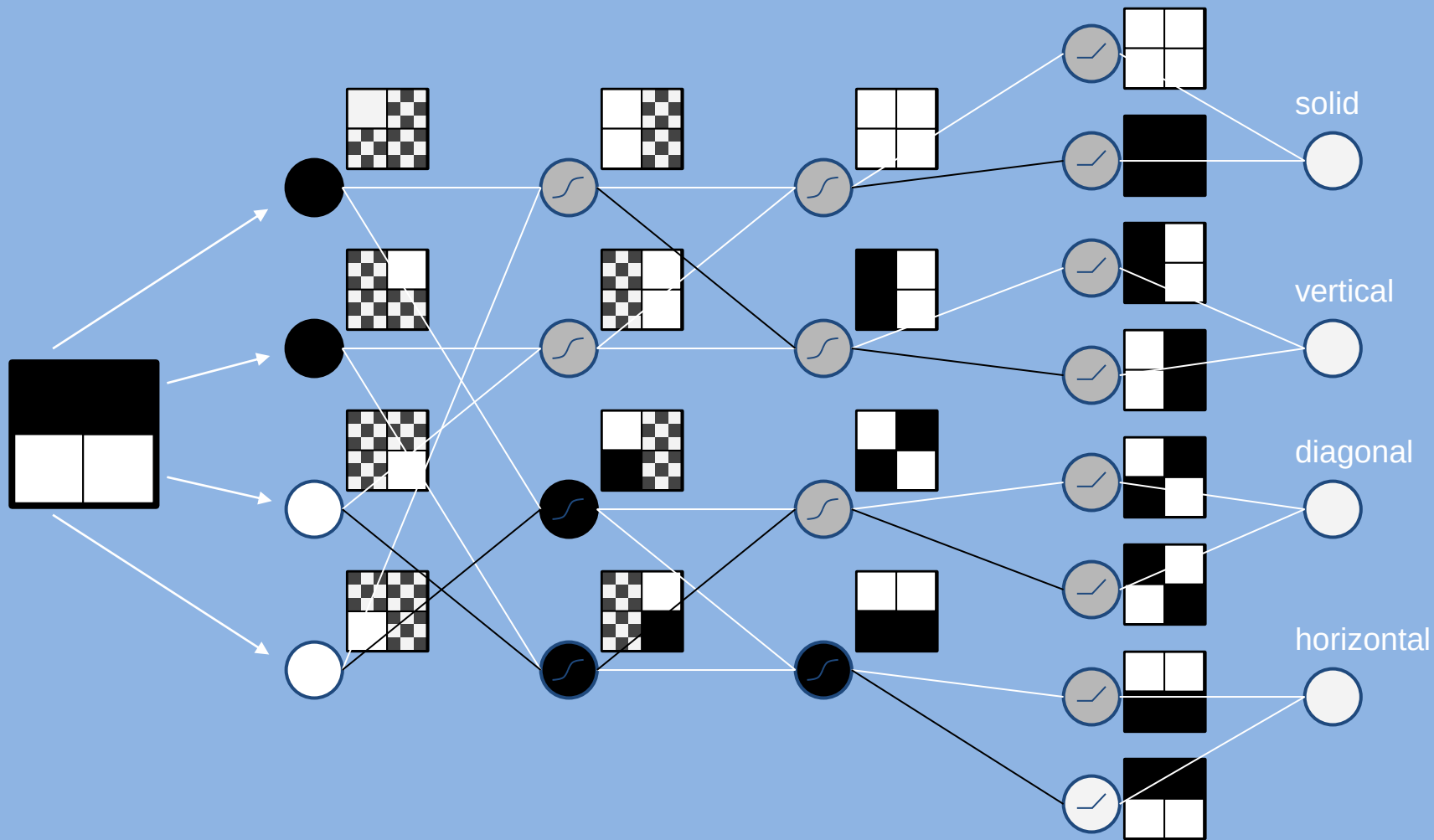


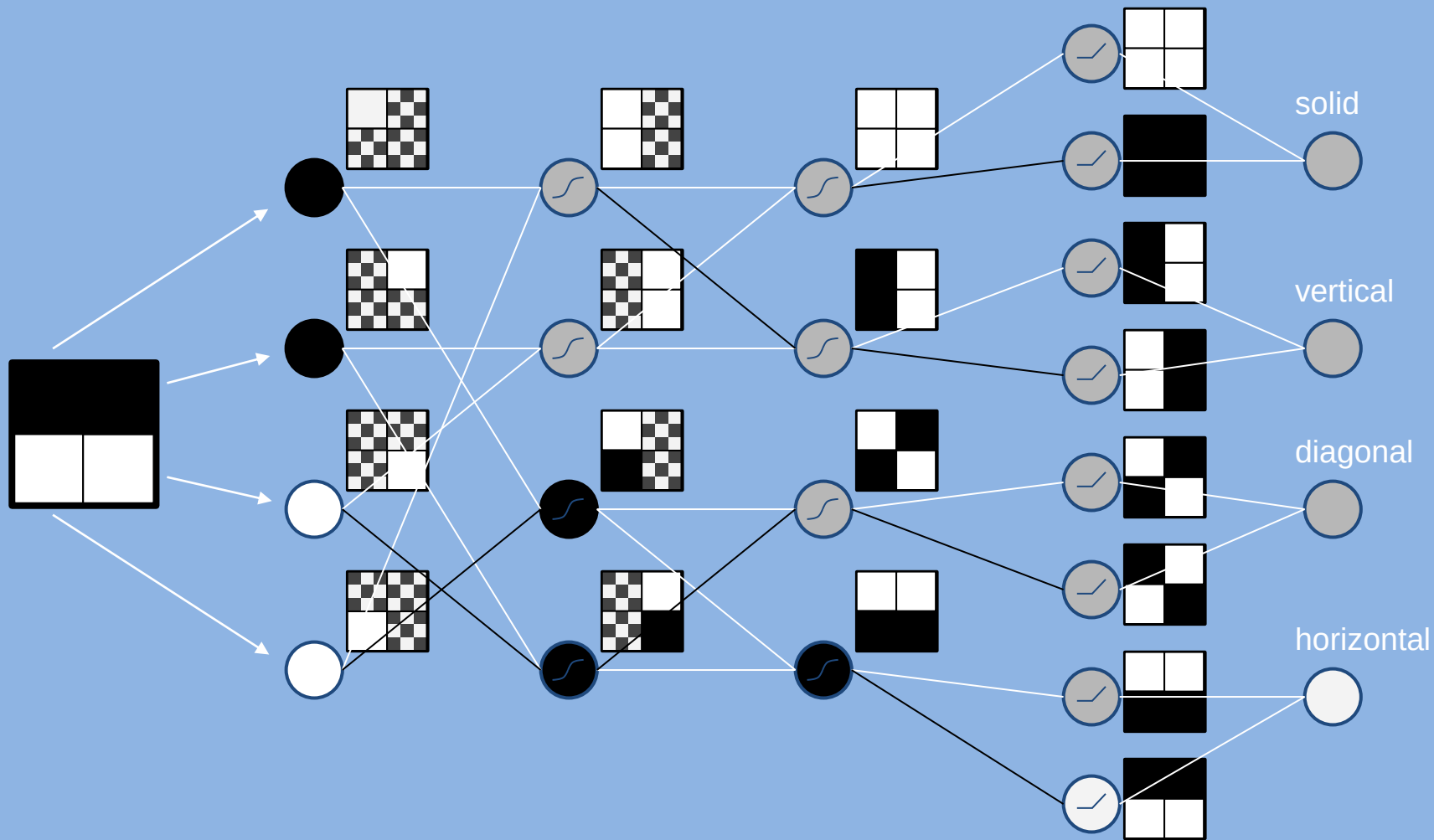


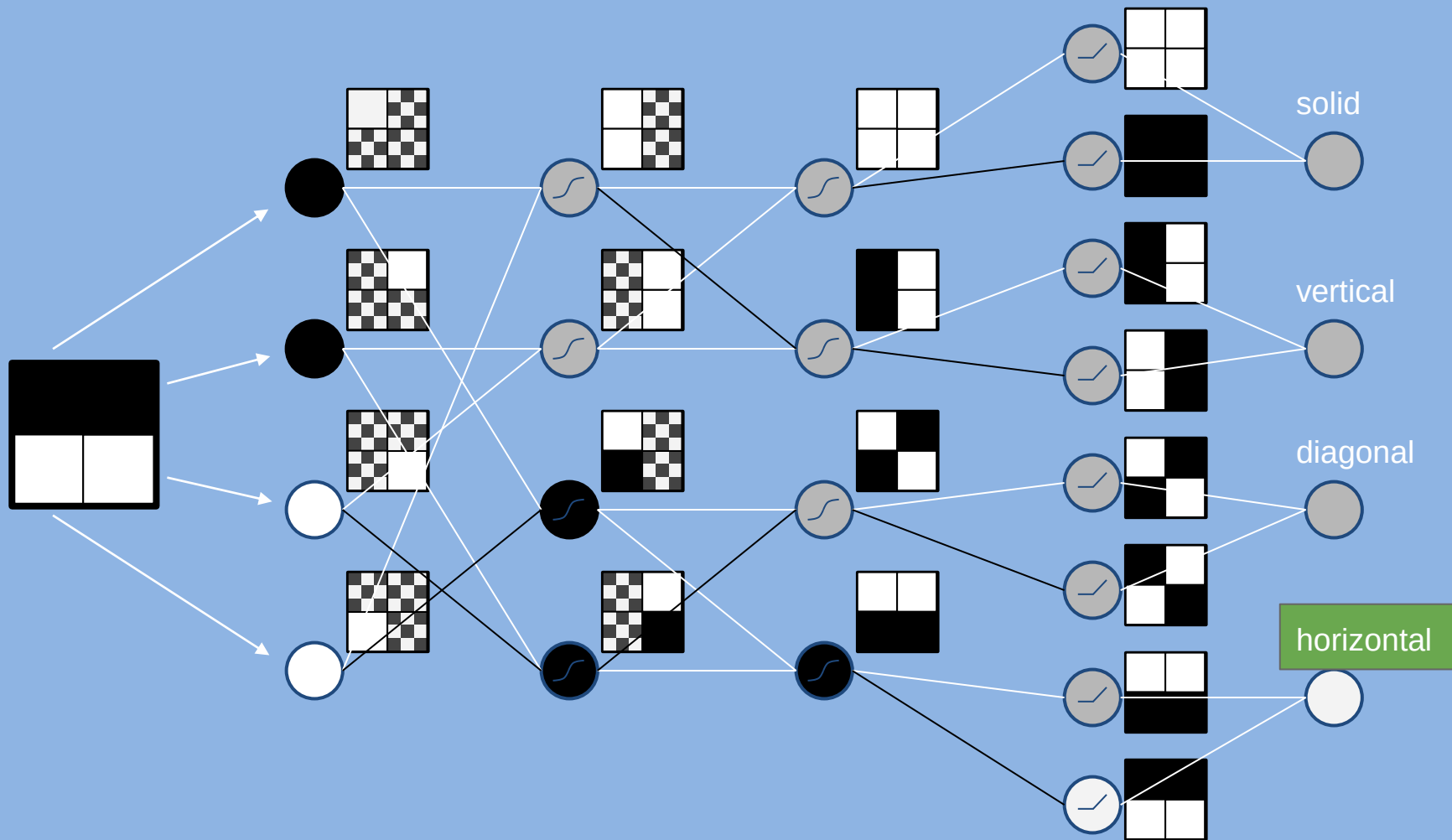




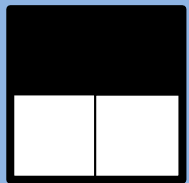








Errors



truth

0.

solid



vertical

0.



diagonal

0.

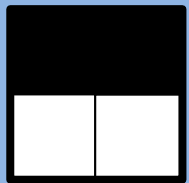


horizontal

1.

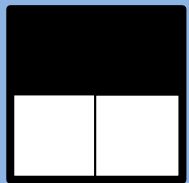


Errors



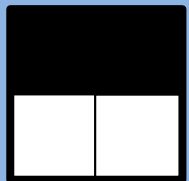
truth	answer	solid
0.	.5	
0.	.75	
0.	-.25	
1.	-.75	
		vertical
		diagonal
		horizontal

Errors



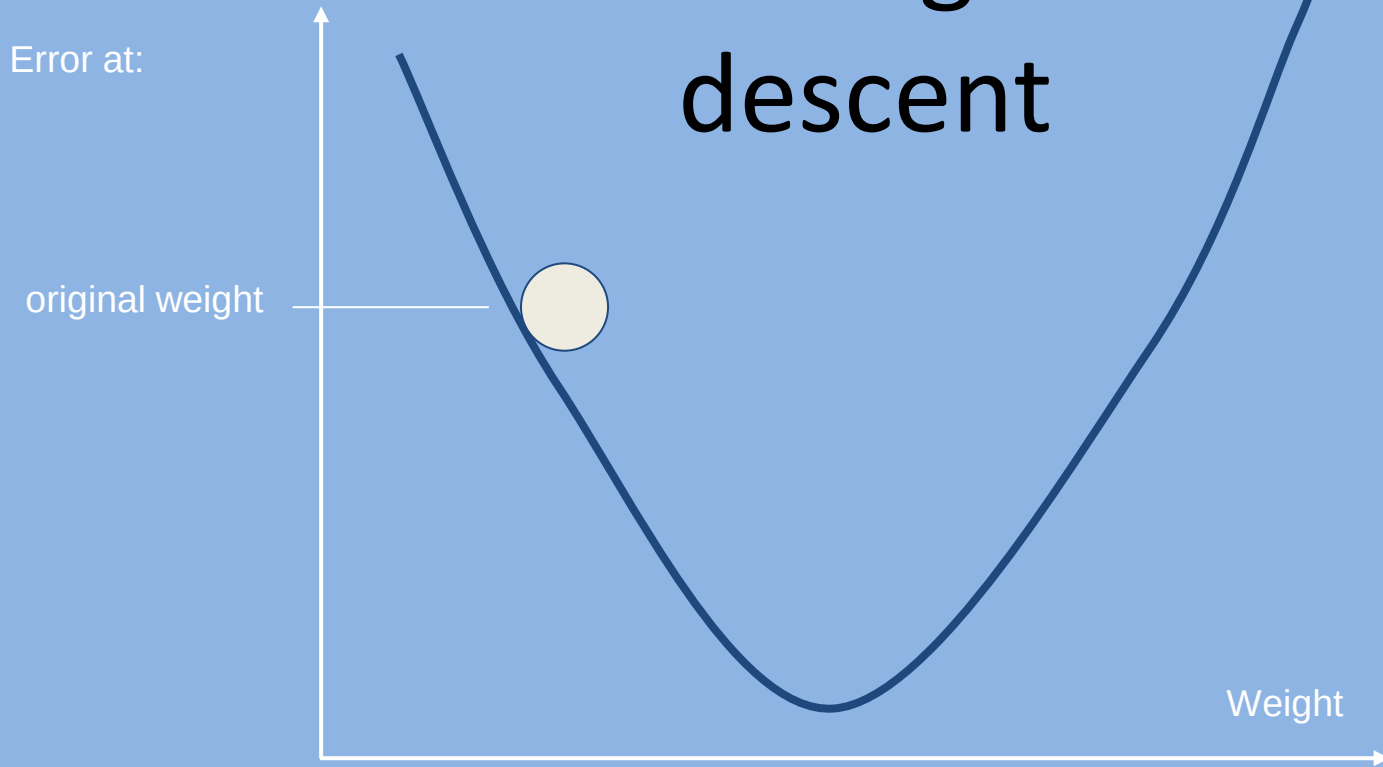
error	truth	answer	solid
.5	0.	.5	
.75	0.	.75	
.25	0.	-.25	
1.75	1.	-.75	
			vertical
			diagonal
			horizontal

Errors

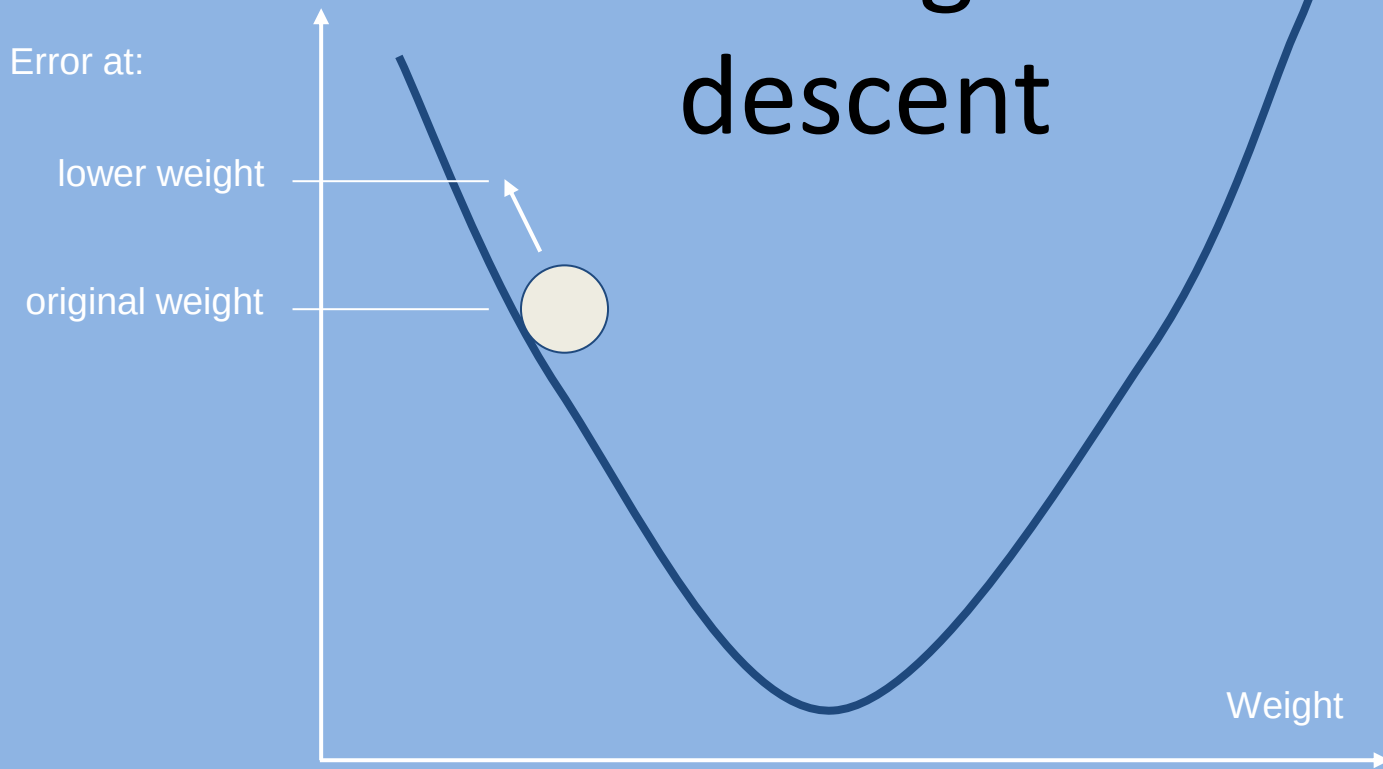


	error	truth	answer	solid
	.5	0.	.5	
	.75	0.	.75	
	.25	0.	-.25	
	1.75	1.	-.75	
total	3.25			

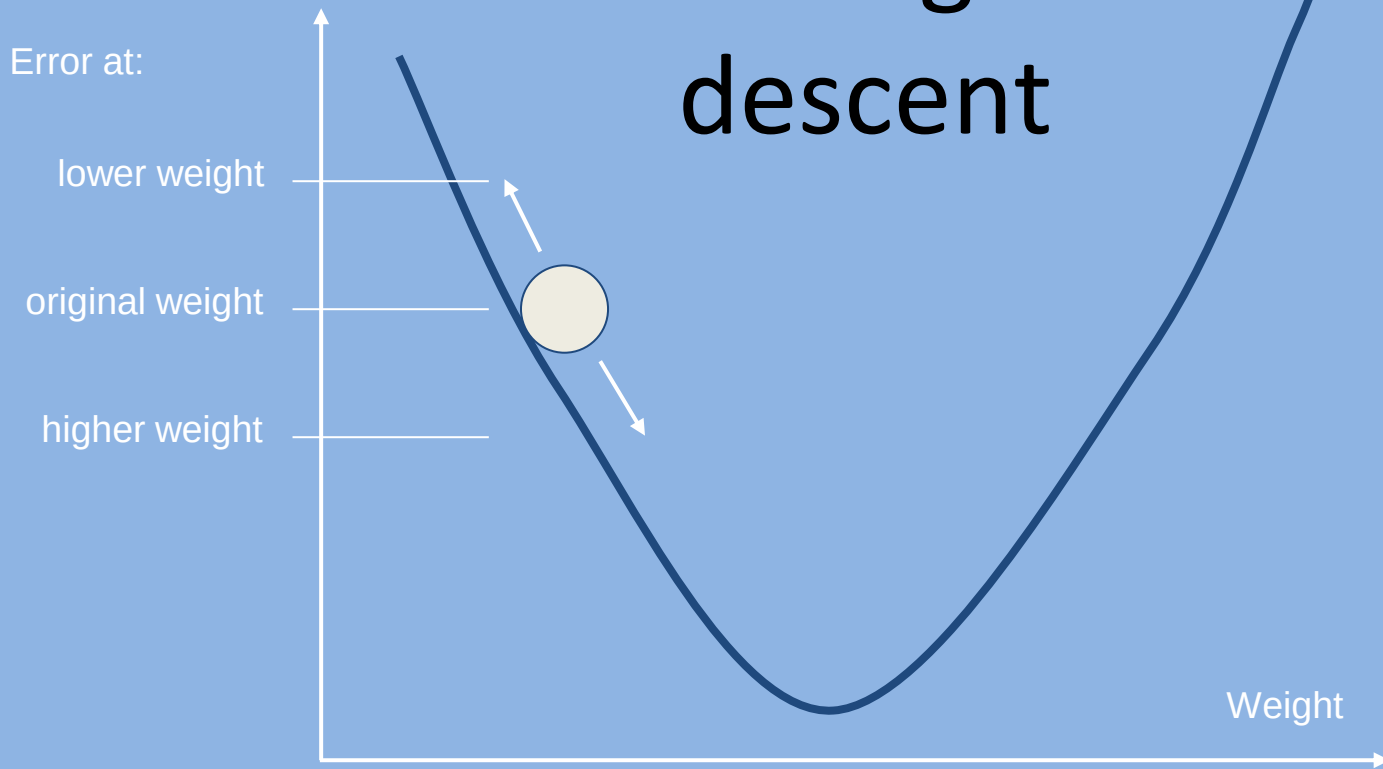
Learn all the weights: Gradient descent



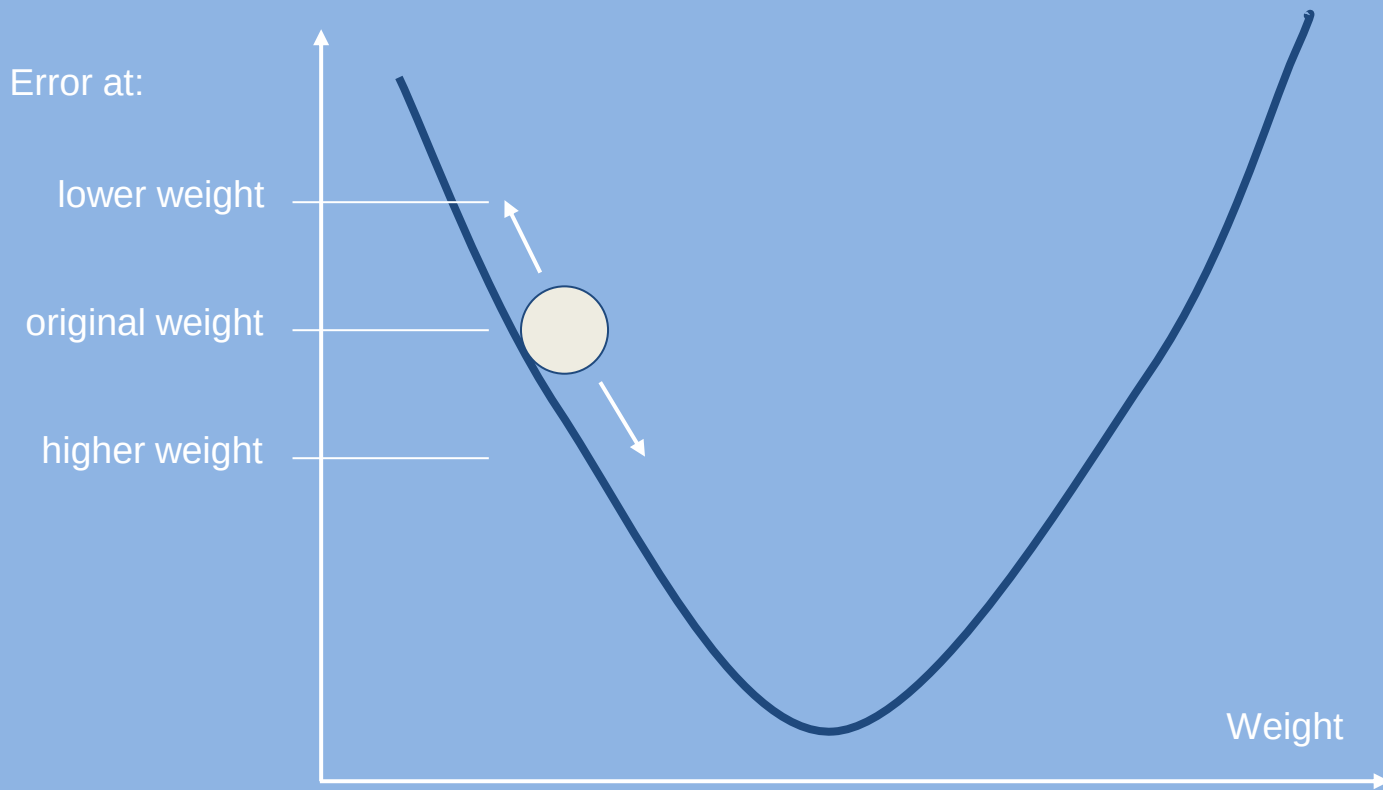
Learn all the weights: Gradient descent



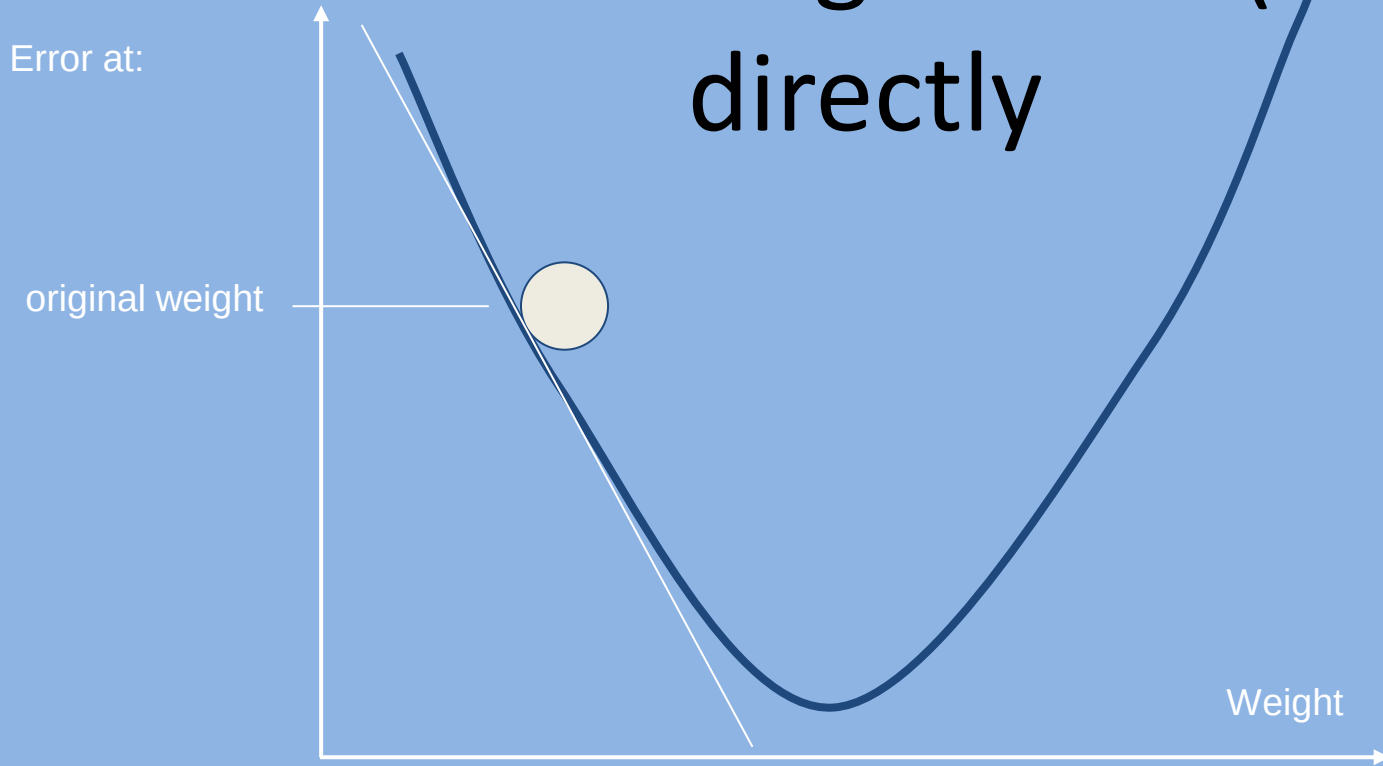
Learn all the weights: Gradient descent



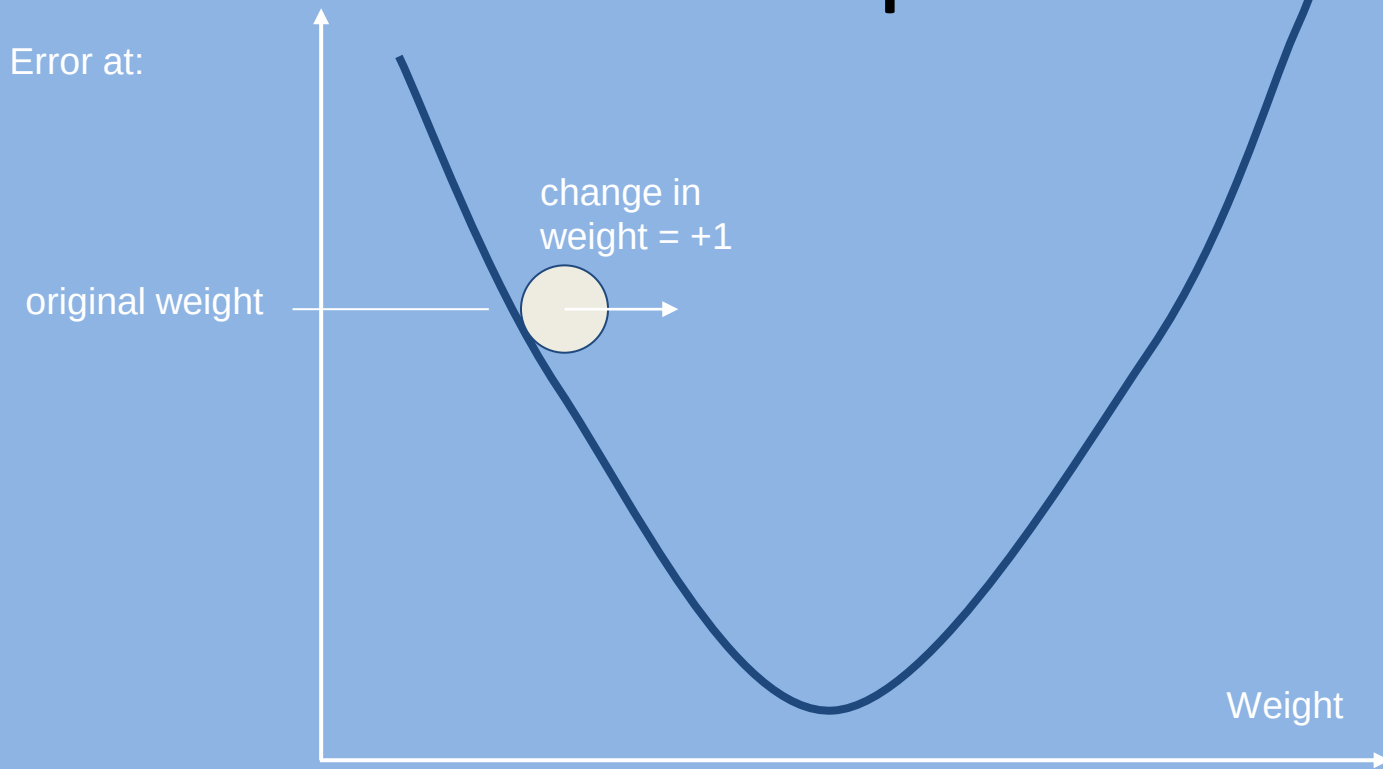
Numerically calculating the gradient is expensive



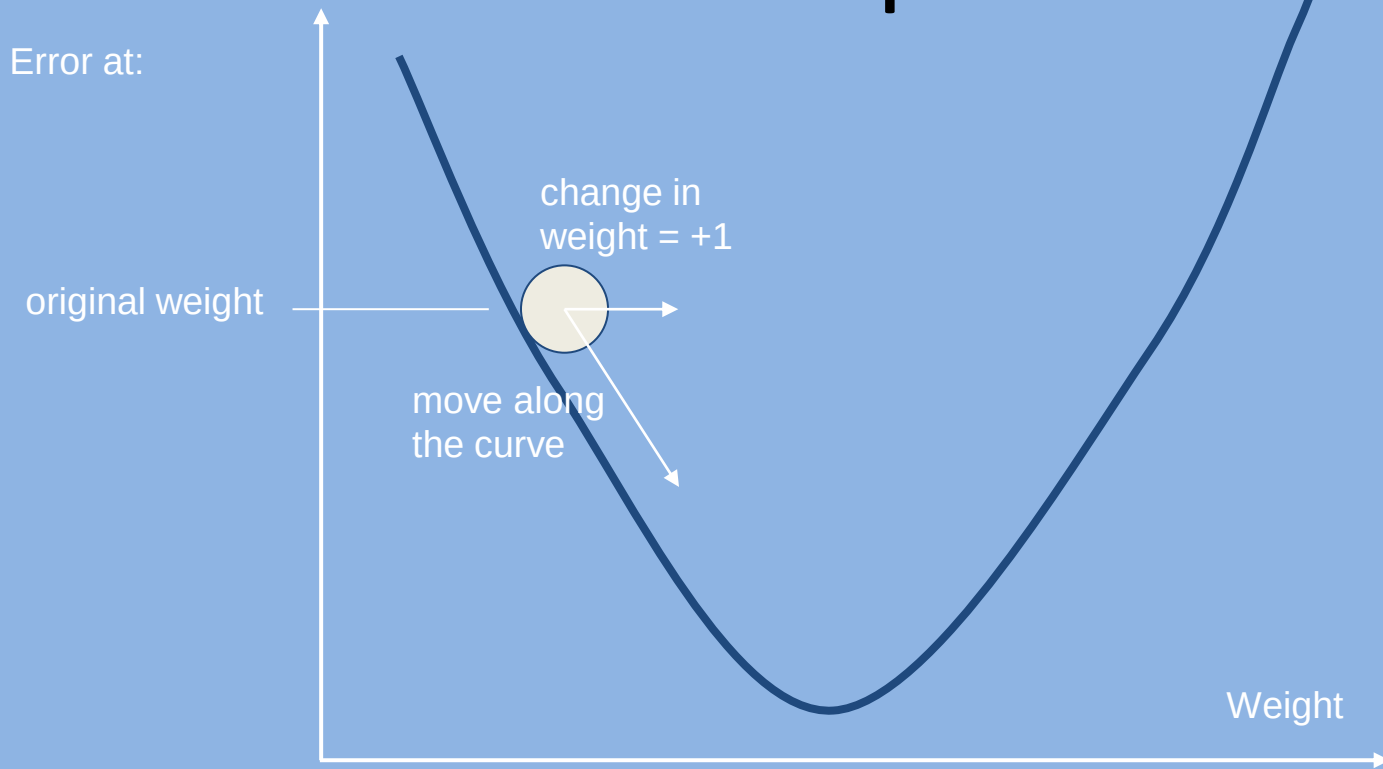
Calculate the gradient (slope) directly



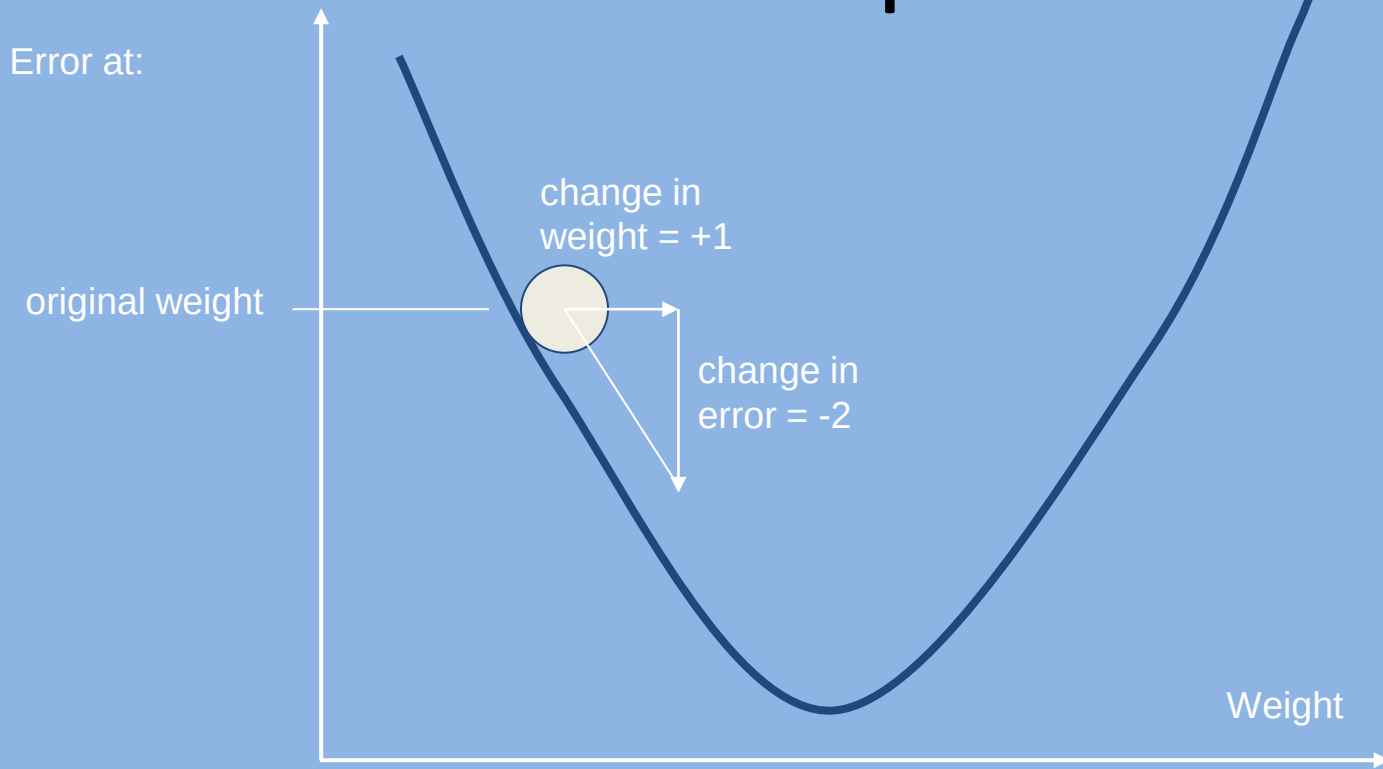
Slope



Slope



Slope



Slope

$$\text{slope} = \frac{\text{change in error}}{\text{change in weight}}$$

